



POLYGON

Agglayer Exit Tool

Security Assessment Report

Version: 2.1

July, 2026

Contents

Introduction	3
Disclaimer	3
Document Structure	3
Overview	3
Security Assessment Summary	4
Scope	4
Approach	4
Coverage Limitations	5
Findings Summary	5
Detailed Findings	6
Summary of Findings	7
External Native-Token Exits Are Replayed As The Local Gas Token	10
Passive ERC-20 Recipients Are Omitted From The Exit Certificate	12
Post-Snapshot L1 Deposits Can Permanently Block The Exit Pipeline	14
Silent Token Balance Fetch Failure In <code>fetchAllTokenBalances</code>	16
Balance Parse Failure Substitutes Zero As Token Budget, Dropping All Bridge Exits For Affected Token	19
Dropped Error In <code>fetchNewWrappedTokenEvents()</code> Produces Silently Incomplete LBT	21
Dropped <code>json.Unmarshal()</code> Error In <code>toAgglayerCertificate()</code> Silently Produces An Empty Exit Certificate	23
Missing Zero-Address Validation On Bridge Contract Binding Produces Silent Incorrect Exit Root	25
<code>BridgeSyncerLite.GetBridges()</code> Unbounded Full-Table Load Causes Out-Of-Memory Crash At Scale	27
Hardcoded <code>dest_net=1</code> In ZkEVM Bridge Service Query Bypasses Configured L2 Network ID	29
Unsettled L2 Bridge Exits Can Block The Exit Pipeline	31
uint64 Addition Overflow In <code>decodeABIString</code> Bypasses Bounds Guard And Crashes The Process	33
<code>checkGenesisBalances()</code> Does Not Query Contract Balances At Genesis Block	35
Duplicate <code>bridgeAsset()</code> Transaction On Post-Delivery Connection Reset Aborts Step G2	36
Exit Tree Built From Non-Transactional Bridge Snapshot	37
Non-Deterministic Last-Write-Wins In <code>overrideMap</code> Silently Records Wrong Sovereign Token Address	39
Non-deterministic Token Ordering In <code>buildSingleEOABalance()</code> Produces Irreproducible Offchain LER	40
Non-Nil Empty LBT Slice Incorrectly Activates Three-Way Comparison	41
Null <code>debug_traceTransaction</code> Result Silently Omits Addresses From Exit Certificate	43
Silent Drop Of <code>BridgeEvent</code> Decode Failures Omits Unclaimed Deposits From Exit Certificate	45
Missing Per-Request Timeout On RPC Log Fetch Can Stall Sync Indefinitely	47
AggregationVKeyHash Field Excluded from <code>AggregationProofPublicValues.Hash()</code>	49
Anvil Prerequisite Check Does Not Respect <code>verifyNewLocalExitRootUsingShadowFork=false</code>	51
<code>batchRPC()</code> Response ID Mismatch Silently Produces Nil Results	52
<code>classifyLogs()</code> Relies On Implicit Correlated Initialisation Of <code>contract</code>	53
<code>hexToUint64</code> Silent Failure On Invalid Input And Overflow	54
Missing Nil Check On <code>r.discarded</code> In ERC-20 Probe Collect Callback	55
Missing Nil Guard On <code>GetCertificateHeader()</code> Return In <code>waitUntilFinal()</code>	56
Missing Nil Guard On <code>RPCExecutionError.Error()</code>	57
Nil Pointer Dereference In <code>sendGroup</code> Debug Log Causes Panic On Native Bridge Exits	58
Nil Pointer Dereference On null JSON Array Element In Bridge Service Responses	59
Non-Deterministic Map Iteration In <code>computeSCLocked()</code> Produces Run-To-Run Variation	60
Non-deterministic Map Iteration In <code>RunStepB3()</code> Produces Non-Reproducible Certificate Outputs	61
No Signal-Aware Context Allows Abrupt Termination Mid-Pipeline	62
Offchain Step G2 Metadata Queries Use <code>latest</code> Block Instead Of Snapshot	64
<code>parseDecimalBigInt()</code> Silently Returns Zero On Parse Failure	65
RPC Error Swallowed In Backward Scan Loop Produces Stale <code>L1InfoTreeLeafCount</code>	66
<code>RunStepA2()</code> Receipt Fetch Failures Silently Swallowed	67
<code>saveJSON</code> Silently Swallows File Write Errors	68
<code>scanBlockHeaders</code> Silently Skips Blocks On JSON Unmarshal Error	69
uint64 Overflow In <code>extractMetadata()</code> Bypasses Bounds Check And Causes Runtime Panic	70

Unclaimed L1-to-L2 Message Deposits Are Excluded From The Exit Certificate	71
Duplicate Sync() And AddBlocks() Methods With AddBlocks() Unused	73
Incomplete --step Usage String Omits Valid Pipeline Steps	75
isEOAResult() Lacks Explicit JSON Null Guard	76
Nil Pointer Dereference In finalizeStepFResult When certificate Is Nil	77
Unbounded Polling Loops With No Timeout Or Signal Handling In Step WAIT	78
Miscellaneous General Comments	79
Step A State Dump Completion Is Inferred From Exhausted Retries	80

A Vulnerability Severity Classification	82
--	-----------

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Polygon components in scope. The review focused solely on the security aspects of these components, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Polygon components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Polygon components in scope.

Overview

The exit certificate tool is a standalone CLI application that generates an Agglayer `Certificate` enabling an L2 chain to exit the Agglayer ecosystem. It executes a multi-step pipeline that scans the entire L2 state from genesis to a configurable target block, collecting all addresses that have interacted with the chain, classifying them as externally-owned accounts or contracts, and computing their ETH and wrapped token balances.

The tool then constructs a certificate containing `BridgeExit` entries that transfer every detected balance to the destination network, including unclaimed L1-to-L2 deposits. It verifies token balances against the Agglayer, computes and validates the `NewLocalExitRoot` (optionally using an Anvil shadow-fork to replay exits against the real bridge contract), and assembles the final signed certificate for submission to the Agglayer. The pipeline comprises prerequisite checks, address discovery via `debug_traceTransaction`, balance aggregation, certificate construction, balance verification, local exit root computation, and certificate signing/submission steps.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [Agglayer](#) repository.

The scope of this time-boxed review was strictly limited to the following files, assessed at commit [3cb76fb](#):

The fixes of the identified issues were assessed at commit [30470c1](#).

The following files were included in the scope of the review:

- `exit_certificate/`:
 - `config.go`
 - `filenames.go`
 - `hex.go`
 - `rpc.go`
 - `run.go`
 - `step_o.go`
 - `step_a.go`
 - `step_b.go`
 - `step_b2.go`
 - `step_b3.go`
- `step_c.go`
- `step_check.go`
- `step_d.go`
- `step_e.go`
- `step_f.go`
- `step_g1.go`
- `step_g_events.go`
- `step_g_order.go`
- `step_h.go`
- `step_i.go`
- `step_sign.go`
- `step_submit.go`
- `types.go`
- `worker.go`
- `cmd/`:
 - `main.go`
- `bridgesyncerlite/`:
 - `downloader.go`
 - `migrations.go`
 - `syncer.go`
 - `types.go`

The changes made in the following PRs were also assessed during the retesting phase:

- [PR#1718](#)
- [PR#1690](#)
- [PR#1727](#)
- [PR#1729](#)

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Golang.

The manual review focused on identifying issues associated with the business logic implementation of the libraries and modules. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Go runtime.

Additionally, the manual review process focused on identifying vulnerabilities related to known Golang anti-patterns and attack vectors, such as integer overflow, floating point underflow, deadlocking, race conditions, memory and CPU exhaustion attacks, and various panic scenarios including `nil` pointer dereferences, index out of bounds, and explicit panic calls.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 49 issues during this assessment. Categorised by their severity:

- High: 4 issues.
- Medium: 7 issues.
- Low: 32 issues.
- Informational: 6 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Polygon components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open**: the finding has not been addressed by the project team.
- **Resolved**: the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- **Closed**: the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
AET-01	External Native-Token Exits Are Replayed As The Local Gas Token	High	Resolved
AET-02	Passive ERC-20 Recipients Are Omitted From The Exit Certificate	High	Resolved
AET-03	Post-Snapshot L1 Deposits Can Permanently Block The Exit Pipeline	High	Resolved
AET-04	Silent Token Balance Fetch Failure In <code>fetchAllTokenBalances</code>	High	Resolved
AET-05	Balance Parse Failure Substitutes Zero As Token Budget, Dropping All Bridge Exits For Affected Token	Medium	Resolved
AET-06	Dropped Error In <code>fetchNewWrappedTokenEvents()</code> Produces Silently Incomplete LBT	Medium	Resolved
AET-07	Dropped <code>json.Unmarshal()</code> Error In <code>toAgglayerCertificate()</code> Silently Produces An Empty Exit Certificate	Medium	Resolved
AET-08	Missing Zero-Address Validation On Bridge Contract Binding Produces Silent Incorrect Exit Root	Medium	Closed
AET-09	<code>BridgeSyncerLite.GetBridges()</code> Unbounded Full-Table Load Causes Out-Of-Memory Crash At Scale	Medium	Closed
AET-10	Hardcoded <code>dest_net=1</code> In ZkEVM Bridge Service Query Bypasses Configured L2 Network ID	Medium	Resolved
AET-11	Unsettled L2 Bridge Exits Can Block The Exit Pipeline	Medium	Resolved
AET-12	<code>uint64</code> Addition Overflow In <code>decodeABIString</code> Bypasses Bounds Guard And Crashes The Process	Low	Resolved
AET-13	<code>checkGenesisBalances()</code> Does Not Query Contract Balances At Genesis Block	Low	Closed
AET-14	Duplicate <code>bridgeAsset()</code> Transaction On Post-Delivery Connection Reset Aborts Step G2	Low	Closed
AET-15	Exit Tree Built From Non-Transactional Bridge Snapshot	Low	Closed
AET-16	Non-Deterministic Last-Write-Wins In <code>overrideMap</code> Silently Records Wrong Sovereign Token Address	Low	Resolved
AET-17	Non-deterministic Token Ordering In <code>buildSingleEOABalance()</code> Produces Irreproducible Offchain LER	Low	Resolved
AET-18	Non-Nil Empty LBT Slice Incorrectly Activates Three-Way Comparison	Low	Closed
AET-19	Null <code>debug_traceTransaction</code> Result Silently Omits Addresses From Exit Certificate	Low	Closed
AET-20	Silent Drop Of <code>BridgeEvent</code> Decode Failures Omits Unclaimed Deposits From Exit Certificate	Low	Resolved
AET-21	Missing Per-Request Timeout On RPC Log Fetch Can Stall Sync Indefinitely	Low	Closed

AET-22	AggregationVKeyHash Field Excluded from AggregationProofPublicValues.Hash()	Low	Closed
AET-23	Anvil Prerequisite Check Does Not Respect verifyNewLocalExitRootUsingShadowFork=false	Low	Closed
AET-24	batchRPC() Response ID Mismatch Silently Produces Nil Results	Low	Resolved
AET-25	classifyLogs() Relies On Implicit Correlated Initialisation Of contract	Low	Closed
AET-26	hexToUint64 Silent Failure On Invalid Input And Overflow	Low	Resolved
AET-27	Missing Nil Check On r.discarded In ERC-20 Probe Collect Callback	Low	Closed
AET-28	Missing Nil Guard On GetCertificateHeader() Return In waitUntilFinal()	Low	Closed
AET-29	Missing Nil Guard On RPCExecutionError.Error()	Low	Closed
AET-30	Nil Pointer Dereference In sendGroup Debug Log Causes Panic On Native Bridge Exits	Low	Closed
AET-31	Nil Pointer Dereference On null JSON Array Element In Bridge Service Responses	Low	Closed
AET-32	Non-Deterministic Map Iteration In computeSCLocked() Produces Run-To-Run Variation	Low	Resolved
AET-33	Non-deterministic Map Iteration In RunStepB3() Produces Non-Reproducible Certificate Outputs	Low	Resolved
AET-34	No Signal-Aware Context Allows Abrupt Termination Mid-Pipeline	Low	Closed
AET-35	Offchain Step G2 Metadata Queries Use latest Block Instead Of Snapshot	Low	Closed
AET-36	parseDecimalBigInt() Silently Returns Zero On Parse Failure	Low	Resolved
AET-37	RPC Error Swallowed In Backward Scan Loop Produces Stale L1InfoTreeLeafCount	Low	Resolved
AET-38	RunStepA2() Receipt Fetch Failures Silently Swallowed	Low	Closed
AET-39	saveJSON Silently Swallows File Write Errors	Low	Resolved
AET-40	scanBlockHeaders Silently Skips Blocks On JSON Unmarshal Error	Low	Closed
AET-41	uint64 Overflow In extractMetadata() Bypasses Bounds Check And Causes Runtime Panic	Low	Closed
AET-42	Unclaimed L1-to-L2 Message Deposits Are Excluded From The Exit Certificate	Low	Closed
AET-43	Duplicate Sync() And AddBlocks() Methods With AddBlocks() Unused	Informational	Closed
AET-44	Incomplete --step Usage String Omits Valid Pipeline Steps	Informational	Closed
AET-45	isEOAResult() Lacks Explicit JSON Null Guard	Informational	Closed
AET-46	Nil Pointer Dereference In finalizeStepFResult When certificate Is Nil	Informational	Closed

AET-47	Unbounded Polling Loops With No Timeout Or Signal Handling In Step WAIT	Informational	Closed
AET-48	Miscellaneous General Comments	Informational	Closed
AET-49	Step A State Dump Completion Is Inferred From Exhausted Retries	Low	Open

AET-01	External Native-Token Exits Are Replayed As The Local Gas Token		
Assets	tools/exit_certificate/step_g2.go tools/exit_certificate/step_g_events.go tools/exit_certificate/step_d.go tools/exit_certificate/step_f.go		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

The exit tool misclassifies bridge exits for external native tokens (tokens whose origin is another network's native asset) as the local gas token during Step G replay. This causes the certificate's `NewLocalExitRoot` to be computed from incorrect bridge leaves, while the tool's internal consistency check still passes because both the shadow-fork replay and the offchain tree builder make the same misclassification.

Step G's `isNativeBridgeExit()` checks only `OriginTokenAddress == zero` without requiring the origin network to match the gas-token network. This conflates two distinct assets:

- the local native gas token, e.g. `(originNetwork=0, originTokenAddress=0x0)`;
- another network's native asset represented on this L2 as a wrapped ERC-20, e.g. `(originNetwork=5, originTokenAddress=0x0)`.

Steps O/B/C/D correctly preserve the full `(originNetwork, originTokenAddress)` identity, and Step F groups by the same pair. Step D builds exits retaining the origin:

```
tools/exit_certificate/step_d.go
exits = append(exits, makeBridgeExit(entry.OriginNetwork, entry.OriginTokenAddress, destNetwork, exitAddr, amount))
```

Step F verifies balances per token identity:

```
tools/exit_certificate/step_f.go
k := tokenKey{exit.TokenInfo.OriginNetwork, exit.TokenInfo.OriginTokenAddress}
```

Step G2 then discards the network component:

```
tools/exit_certificate/step_g2.go::isNativeBridgeExit()
func isNativeBridgeExit(ti *agglayer.TokenInfo, gasTokenNetwork uint32, gasTokenAddress common.Address) bool {
    return ti == nil ||
        ti.OriginTokenAddress == (common.Address{}) ||
        (ti.OriginNetwork == gasTokenNetwork && ti.OriginTokenAddress == gasTokenAddress)
}
```

The second condition ignores `OriginNetwork`, so any zero-address origin token is treated as native. This drives the shadow-fork replay to send `bridgeAsset` with `token=0x0` and `msg.value=amount`:

```
tools/exit_certificate/step_g2.go
isNative := isNativeBridgeExit(bridge.TokenInfo, gasTokenNetwork, gasTokenAddress)
...
if isNative && bridgeExit.Amount != nil {
    value = bridgeExit.Amount
}
callData := encodeBridgeAssetCallRaw(..., l2TokenAddr)
```

The offchain tree builder repeats the same mistake, defaulting to the gas token when `OriginTokenAddress` is zero:

```
tools/exit_certificate/step_g_events.go
originNetwork, originAddr := gasTokenNetwork, gasTokenAddress
if be.TokenInfo != nil && be.TokenInfo.OriginTokenAddress != (common.Address{}) {
    originNetwork = be.TokenInfo.OriginNetwork
    originAddr = be.TokenInfo.OriginTokenAddress
}
```

Because both paths apply the same flawed classification, Step G's `treeRoot == shadowForkLER` check passes despite encoding the wrong asset into the exit tree.

This issue has a high impact as the certificate's exit root is computed from bridge leaves that encode the wrong asset identity, meaning onchain verification would process exits against an incorrect tree. This issue has a medium likelihood as it requires a wrapped native token from another network to exist on the exiting L2 with non-zero supply — a condition that depends on the specific chain's token ecosystem.

Recommendations

Make native classification require an exact gas-token identity match:

```
func isNativeBridgeExit(ti *agglayerTypes.TokenInfo, gasTokenNetwork uint32, gasTokenAddress common.Address) bool {
    if ti == nil {
        return true
    }
    return ti.OriginNetwork == gasTokenNetwork && ti.OriginTokenAddress == gasTokenAddress
}
```

For standard ETH gas-token chains, this still treats `(0, 0x0)` as native. It no longer treats `(N, 0x0)` as native when `N != gasTokenNetwork`.

Resolution

The issue has been fixed in [PR#1698](#). The recommendation was implemented.

AET-02 Passive ERC-20 Recipients Are Omitted From The Exit Certificate

Assets

```
tools/exit_certificate/step_a.go
tools/exit_certificate/step_b.go
tools/exit_certificate/step_c.go
tools/exit_certificate/step_d.go
tools/exit_certificate/step_f.go
```

Status **Resolved:** See [Resolution](#)

Rating

Severity: High

Impact: High

Likelihood: Medium

Description

ERC-20 holders that only received tokens passively (without ever sending a transaction or otherwise modifying their account state) are excluded from the exit certificate. Their balances are absorbed into the smart-contract-locked residual and routed to `exitAddress`, causing direct recipient corruption that passes aggregate balance verification.

Step A discovers candidate holders by tracing transactions with Geth's `prestateTracer` in `diffMode`, then records only the top-level account keys from `pre` and `post`:

```
tools/exit_certificate/step_a.go

result, err := singleRPC(ctx, rpcURL, "debug_traceTransaction", []any{
    txHash.Hex(),
    map[string]any{
        "tracer": "prestateTracer",
        "tracerConfig": map[string]any{"diffMode": true},
    },
}, defaultRetries)

for addr := range trace.Pre {
    addrSet[common.HexToAddress(addr)] = struct{}{}
}
for addr := range trace.Post {
    addrSet[common.HexToAddress(addr)] = struct{}{}
}
```

For an ERC-20 transfer, the receiver's token balance is stored in the token contract's storage. The receiver account itself does not need a nonce, native balance change, code change, or log emission. Geth's `prestateTracer` diff output is keyed by modified accounts, and token balance updates modify the token contract account, not the recipient account. As a result, Step A can successfully trace every transaction while never adding the token recipient to the address set.

`RunStepA2()` does not repair this gap — it is only called for failed traces and recovers sender, recipient, created-contract, and log-emitter addresses, but not addresses encoded inside ERC-20 `Transfer` topics or token storage slots.

Step B queries ERC-20 balances only for the Step A EOA set:

```
tools/exit_certificate/step_b.go

tokenBalances := fetchAllTokenBalances(ctx, rpcURL, stepA.WrappedTokens, eoaAddrs, blockTag, batchSize, concurrency)
```

Step C then computes `SC_locked = LBT_totalSupply - accumulated_EOA_balances`, so the missed holder's balance is absorbed into the smart-contract-locked value:

```
tools/exit_certificate/step_c.go

locked := new(big.Int).Sub(lbtBalance, eoaTotal)
```

Step D routes the SC-locked value to `exitAddress`:

```
tools/exit_certificate/step_d.go
exits = append(exits, makeBridgeExit(entry.OriginNetwork, entry.OriginTokenAddress, destNetwork, exitAddr, amount))
```

Step F groups by token only, comparing aggregate totals against LBT/agglayer balances. It does not validate that per-holder recipients match the L2 token holders, so the certificate passes aggregate verification while assigning a passive holder's tokens to `exitAddress`.

A concrete failure path:

1. Alice holds a wrapped ERC-20 in an active account already discovered by Step A.
2. Alice transfers tokens to Bob's cold address.
3. Bob never sends a transaction or otherwise touches state before the target block.
4. The ERC-20 transfer modifies the token contract storage, so Step A includes the token contract but not Bob.
5. Step B never calls `balanceOf(token, Bob)`.
6. Step C absorbs Bob's balance into `SC_locked`; Step D routes it to `exitAddress`.
7. Step F passes because the aggregate token amount is conserved.

This issue has a high impact as affected holders lose their individual bridge exits and their balances are permanently assigned to `exitAddress`. Recovery depends on whoever controls `exitAddress` and any offchain redistribution process. This issue has a medium likelihood as passive ERC-20 recipients (cold wallets, exchange deposit addresses, treasury addresses) are common and the condition can be created permissionlessly by transferring wrapped tokens to a fresh address.

Recommendations

Do not rely on account-state traces as the only ERC-20 holder source. For every wrapped token in the LBT, scan `Transfer(address,address,uint256)` logs up to `targetBlock` and add both indexed participants to the balance-query set before Step B runs.

```
for _, log := range transferLogs {
    from := common.BytesToAddress(common.FromHex(log.Topics[1])[12:])
    to := common.BytesToAddress(common.FromHex(log.Topics[2])[12:])
    if from != (common.Address{}) {
        addCandidate(from)
    }
    if to != (common.Address{}) {
        addCandidate(to)
    }
}
```

Fail closed if any token log range cannot be scanned or decoded. Add a regression test where a wrapped ERC-20 is transferred to a never-active address and assert that the recipient receives an individual bridge exit.

Resolution

The issue has been fixed in [PR#1701](#). Address discovery was updated to discover passive token-only holders.

AET-03	Post-Snapshot L1 Deposits Can Permanently Block The Exit Pipeline		
Assets	tools/exit_certificate/step_e.go tools/exit_certificate/run.go tools/exit_certificate/README.md		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

An unprivileged user can permanently block exit certificate generation by submitting a dust L1-to-L2 asset deposit after the sequencer is stopped. Step E scans L1 up to the current latest block rather than using a cutoff tied to the frozen L2 snapshot, so post-snapshot deposits are picked up as unclaimed and abort the pipeline by default.

The `README.md` requires the sequencer to be stopped before running the tool:

```
Halt the sequencer before running the tool so that no new bridges (or other state changes) are produced while the certificate is
↪ being built.
```

Step O resolves the L2 `targetBlock` once and all subsequent L2 scans use that fixed block. Step E has no equivalent L1 end block — it resolves the latest L1 block at runtime:

```
tools/exit_certificate/step_e.go
l1LatestBlock, err := resolveL1LatestBlock(ctx, cfg)
...
l1Deposits, err := fetchL1BridgeEvents(ctx, cfg, l1LatestBlock)
```

`resolveL1LatestBlock()` always reads `eth_blockNumber` from L1:

```
tools/exit_certificate/step_e.go
latestResult, err := singleRPC(ctx, cfg.L1RPCURL, "eth_blockNumber", nil, defaultRetries)
...
block := hexToUint64(latestHex)
```

`fetchL1BridgeEvents()` then scans from `l1StartBlock` through that latest block:

```
tools/exit_certificate/step_e.go
for start := fromBlock; start <= l1LatestBlock; start += uint64(blockRange) {
    end := min(start+uint64(blockRange)-1, l1LatestBlock)
    jobs = append(jobs, blockRangeJob{from: start, to: end})
}
```

For every deposit found, Step E calls `isClaimed()` on the L2 bridge at `"latest"`:

```
tools/exit_certificate/step_e.go
calls[i] = RPCCall{
    Method: "eth_call",
    Params: []any{
        map[string]string{
            "to":    cfg.L2BridgeAddress.Hex(),
            "data":  encodeIsClaimed(dep.DepositCount, sourceBridgeNetworkMainnet),
        },
        "latest",
    },
}
```

A deposit created after the L2 target block cannot be part of the frozen L2 state, and with the sequencer stopped it cannot be claimed. Step E therefore classifies it as unclaimed, which is fatal by default:

```
tools/exit_certificate/step_e.go
if len(unclaimedAssets) == 0 {
    return &StepEResult{FinalCertificate: certificate}, nil
}
if cfg.Options.IgnoreUnclaimed {
    return &StepEResult{FinalCertificate: certificate}, nil
}
return &StepEResult{FinalCertificate: nil}, fmt.Errorf(
    "unclaimed deposits not supported ...: %d unclaimed asset deposit(s)",
    len(unclaimedAssets),
)
```

An attacker can exploit this by watching for an announced exit or stopped sequencer, then submitting a minimal L1 asset deposit before Step E completes. Restarting the sequencer to claim the deposit changes the L2 state and requires rebuilding the exit; the attacker can repeat the same dust deposit after each new snapshot.

The global `ignoreUnclaimed=true` switch is not a clean mitigation because it suppresses all unclaimed asset deposits, including legitimate pre-snapshot deposits that should block or be handled explicitly.

This issue has a high impact as it enables a permissionless liveness failure for the chain exit, forcing either an unsafe global ignore or repeated snapshot restarts. This issue has a medium likelihood as L1-to-L2 deposits are permissionless and the attack only requires submitting a minimal asset deposit while the sequencer is stopped.

Recommendations

Add an explicit L1 scan cutoff for Step E and require operators to capture it before or at the same time the L2 snapshot is finalised. Step E should scan only `l1StartBlock..l1EndBlock`, not the L1 latest block at runtime.

```
if cfg.Options.L1EndBlock == 0 {
    return nil, fmt.Errorf("l1EndBlock is required for Step E")
}
l1Deposits, err := fetchL1BridgeEvents(ctx, cfg, cfg.Options.L1EndBlock)
```

Post-cutoff L1 deposits should be reported separately as out-of-snapshot deposits and must not block the certificate for the frozen L2 target. Keep `ignoreUnclaimed` limited to deposits inside the explicit Step E scan window.

Resolution

The issue has been fixed in [PR#1704](#). A new option `options.l1EndBlock` was added.

AET-04 Silent Token Balance Fetch Failure In <code>fetchAllTokenBalances</code>			
Assets	tools/exit_certificate/step_b.go		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

When any wrapped token's `balanceOf` batch fails inside `step_b.fetchAllTokenBalances()`, the error is silently discarded. The affected token is absent from the returned balance map, causing Step C to treat its entire LBT supply as SC-locked value. The final certificate sends the whole token supply to `exitAddress` rather than distributing it across individual EOA holders, excluding those holders from the certificate entirely.

`step_b.fetchAllTokenBalances()` spawns one goroutine per wrapped token to fetch `balanceOf` balances for all EOA addresses. The function signature returns only a map — there is no error return. When a goroutine's call to `step_b.fetchTokenBalances()` fails (for example due to an RPC timeout, rate-limit, or connection reset), the goroutine logs a `Warnf` message and returns without writing to the shared `tokenBalances` map. The caller, `step_b.RunStepB1()`, receives a map that silently omits the failed token:

```
func fetchAllTokenBalances(
    ctx context.Context, rpcURL string, tokens []WrappedToken,
    eoaAddresses []common.Address, blockTag string, batchSize, concurrency int,
) map[common.Address]map[common.Address]*big.Int { // @audit: no error return – failures are hidden from callers

    var mu sync.Mutex
    tokenBalances := make(map[common.Address]map[common.Address]*big.Int)
    sem := make(chan struct{}, tokenConcurrency)

    var wg sync.WaitGroup
    for _, token := range tokens {
        wg.Add(1)
        sem <- struct{}{}
        go func(tok WrappedToken) {
            defer wg.Done()
            defer func() { <-sem }()

            balances, err := fetchTokenBalances(
                ctx, rpcURL, tok.WrappedTokenAddress,
                eoaAddresses, blockTag, batchSize, concurrency,
            )
            if err != nil {
                // @audit: error silently swallowed – only a warning is logged; token absent from map
                log.Warnf("Failed to fetch balances for token %s: %v", tok.WrappedTokenAddress.Hex(), err)
                return // @audit: tokenBalances[tok] is never set; token is missing from the output
            }
            if len(balances) > 0 {
                mu.Lock()
                tokenBalances[tok.WrappedTokenAddress] = balances
                mu.Unlock()
            }
        }(token)
    }
    wg.Wait()
    return tokenBalances // @audit: may silently omit entries for tokens whose fetch failed
}
```

`step_b.RunStepB1()` passes the returned map directly to `step_b.buildAccumulated()` with no check for missing tokens. `step_b.buildAccumulated()` only iterates keys present in the map, so the failed token produces no `AccumulatedBalance` entry. Step C then computes the SC-locked value per token using:

```
// step_c.go - computeSCLocked
for tokenKey, lbt := range lbtByToken {
    lbtBalance := parseDecimalBigInt(lbt.Balance)
    eoaTotal := new(big.Int)
    if val, exists := eoaByToken[tokenKey]; exists {
        eoaTotal.Set(val)
    }
    // @audit: when the token is absent from eoaByToken, eoaTotal stays zero
    locked := new(big.Int).Sub(lbtBalance, eoaTotal)
    // locked = lbtBalance - the entire token supply is sent to exitAddress
}
```

With `eoatotal` defaulting to zero, `locked` equals the full LBT total supply for the affected token. Step D then creates a single `BridgeExit` routing the entire supply to `exitAddress`. Every individual EOA that holds a balance of that token is excluded from the certificate — their assets are effectively consolidated under the exit address rather than being claimable at the destination chain.

The Step F balance check cannot detect this error. Step F performs a three-way comparison of `LBT total == agglayer balance == certificate sum per token`. When the entire supply is routed to `exitAddress`, the certificate sum per token still equals the LBT total, so the comparison passes. The distribution error is invisible to the check.

`step_b.fetchTokenBalances()` calls `rpc.concurrentBatchRPC()` internally, which retries up to three times (defaultRetries=3) before returning an error. This reduces but does not eliminate the likelihood of a failure reaching `step_b.fetchAllTokenBalances()` — a persistently rate-limited or degraded RPC node will exhaust all retries. The contrasting pattern in Step B2 (`step_b2.RunStepB2()`) correctly propagates errors from `rpc.concurrentBatchRPC()` with `return nil, fmt.Errorf(...)`, making `step_b.fetchAllTokenBalances()` the only place in the pipeline where per-token RPC errors are silently discarded.

This issue has a high impact because, when triggered, individual EOA holders of the affected token are entirely absent from the exit certificate. Their assets are instead assigned to `exitAddress` as SC-locked value. The aggregate value per token is conserved, but the distribution is incorrect: token holders cannot claim their assets individually at the destination chain. This issue has a medium likelihood because transient RPC failures are a routine occurrence under batch-heavy workloads against remote archive nodes, and the exit procedure is precisely the scenario where an RPC node may be under load.

Recommendations

Change `step_b.fetchAllTokenBalances()` to return an error alongside the balance map. Collect all per-token errors across goroutines (under the existing mutex) and return a combined error after `wg.Wait()` if any goroutine failed. Update `step_b.RunStepB1()` to check the returned error and abort the step before calling `step_b.buildAccumulated()`.

The updated signature should be:

```
func fetchAllTokenBalances(...) (map[common.Address]map[common.Address]*big.Int, error)
```

Goroutine failures should append to a shared error slice under the mutex, and the function should return a descriptive combined error (for example, using `errors.Join`) when the slice is non-empty after all goroutines complete.

Update the call site in `step_b.RunStepB1()` to propagate the returned error:

```
tokenBalances, err := fetchAllTokenBalances(ctx, rpcURL, stepA.WrappedTokens, eoaAddrs, blockTag, batchSize, concurrency)
if err != nil {
    return nil, fmt.Errorf("fetch token balances: %w", err)
}
```

Resolution

The issue has been fixed in [PR#1675](#). The error is now propagated.

AET-05	Balance Parse Failure Substitutes Zero As Token Budget, Dropping All Bridge Exits For Affected Token		
Assets	tools/exit_certificate/step_f.go		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

When `big.Int.SetString()` fails to parse the balance string for a token entry in the Agglayer admin API response or the LBT file, the affected token is silently omitted from the balance map. Because the comparison loop sources its iteration keys from all three data sources — including the certificate's own exit groups — the token is still visited during the comparison, at which point a nil map lookup substitutes zero as the reference balance. With `ignoreBalanceMismatch=true`, this zero budget is propagated directly to `capCertificateExits()`, which drops every bridge exit for that token from the certificate. The affected token's transfers are permanently removed from the exit certificate, and the corresponding funds cannot be recovered by the users whose exits were dropped.

`compareTokenBalances()` (for the Agglayer admin code path) and `compareCertificateToLBT()` (for the offline LBT code path) both follow the same pattern. When `new(big.Int).SetString(e.Amount, decimalBase)` returns `(nil, false)`, a warning is logged and the loop continues, leaving the token key absent from the respective balance map (`agglayerMap` or `lbtMap`). The `seen` set that drives the subsequent comparison loop is constructed from the union of all three maps — certificate exit groups, the Agglayer map, and the LBT map — so a token present in the certificate will still enter the comparison loop regardless of whether its external balance was successfully parsed.

```
// agglayer parse failure path
for _, e := range agglayerEntries {
    k := tokenKey{e.OriginNetwork, e.OriginTokenAddress}
    amount, ok := new(big.Int).SetString(e.Amount, decimalBase)
    if !ok {
        log.Warnf("Could not parse agglayer amount %q for token (network=%d addr=%s)",
            e.Amount, e.OriginNetwork, e.OriginTokenAddress.Hex())
        continue // @audit token key NOT inserted into agglayerMap
    }
    agglayerMap[k] = amount
}
// ... seen is built from groups, agglayerMap, AND lbtMap ...
for k := range seen {
    agglAmt := agglayerMap[k]
    if agglAmt == nil {
        agglAmt = new(big.Int) // @audit zero substituted for missing agglayer balance
    }
    // ...
    if agglAmt.Cmp(lbtAmt) <= 0 {
        check.RemainingBalance = new(big.Int).Set(agglAmt) // @audit RemainingBalance = 0
    } else {
        check.RemainingBalance = new(big.Int).Set(lbtAmt) // @audit RemainingBalance = 0 (lbtAmt also 0 when absent)
    }
}
```

The same nil-substitution pattern applies in `compareCertificateToLBT()` for the LBT map. When an LBT entry's balance string cannot be parsed, `lbtMap[k]` is nil at comparison time and `lbtAmt` is set to `new(big.Int)` (zero), producing the same zero `RemainingBalance`.

The consequence depends on the `ignoreBalanceMismatch` configuration. With the default setting of `false`, the zero Agglayer or LBT amount produces a mismatch against the non-zero certificate sum, which causes `finalizeStepFResult()` to return an error and abort the pipeline. This is a denial-of-service against the exit pipeline for a parse error in an external data source: a single malformed balance string from the Agglayer admin API halts the entire exit process. With `ignoreBalanceMismatch=true`, the same mismatch activates the capping logic. `capCertificateExits()` receives

`RemainingBalance=0` for the affected token and, at line [418], treats a zero budget as an instruction to drop all exits for that token entirely. The complete trace is: `RunStepF()` → `json.Unmarshal()` succeeds with structurally valid JSON but a non-decimal `Amount` field → `compareTokenBalances()` → `SetString()` fails → key absent from `aggLayerMap` → nil lookup → `agglAmt=0` → `RemainingBalance=0` → `capCertificateExits()` drops all exits → capped certificate missing all exits for the token.

The Agglayer admin API documents the balance field as a decimal `U256` string. A malformed value (an empty string, a hex-encoded amount such as `"0x1a"`, or a response resulting from an API schema change) is the primary trigger for the Agglayer path. For the LBT path, the `Balance` field is populated by `big.Int.String()` in Step 0 from onchain data and will always produce a valid decimal under normal circumstances; file corruption or manual editing are the realistic triggers there.

This issue has a high impact in the `ignoreBalanceMismatch=true` configuration path because all bridge exits for the affected token are silently removed from the certificate. In the context of a chain exit, omitting bridge exits means those token balances are not transferred to the destination network and may not be recoverable by the affected users. The damage may be permanent once the certificate is submitted and settled.

This issue has a low likelihood because it requires a malformed balance string to arrive from the Agglayer admin API or to be present in the LBT file. Under normal operation both sources produce well-formed decimal strings. An API regression, schema change, or file corruption is required to trigger the defect.

Recommendations

Return an error from `compareTokenBalances()` and `compareCertificateToLBT()` immediately when any entry's balance string cannot be parsed, rather than logging a warning and continuing:

```
amount, ok := new(big.Int).SetString(e.Amount, decimalBase)
if !ok {
    return nil, fmt.Errorf("could not parse agglayer amount %q for token (network=%d addr=%s)",
        e.Amount, e.OriginNetwork, e.OriginTokenAddress.Hex())
}
aggLayerMap[k] = amount
```

Apply the same change to the corresponding LBT parsing loop in both `compareTokenBalances()` and `compareCertificateToLBT()`. This ensures that any malformed external data causes an immediate, clearly attributed pipeline failure rather than propagating silently to a zero-budget cap that drops exits. An operator who sees an explicit parse error can investigate the Agglayer response or LBT file and correct the root cause before resubmitting the exit certificate.

Resolution

The issue has been fixed in [PR#1716](#). The error is now propagated.

AET-06	Dropped Error In <code>fetchNewWrappedTokenEvents()</code> Produces Silently Incomplete LBT		
Assets	tools/exit_certificate/step_0.go		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

When `fetchNewWrappedTokenEvents()` encounters an RPC failure during event scanning, the error is silently discarded and a partial token list is returned as if the scan had succeeded. Because the function has no error return value, the caller cannot detect the failure, and the pipeline continues with an incomplete Local Balance Tree (LBT). Tokens absent from the LBT are omitted from the exit certificate, leaving affected token holders with no bridge exits and no mechanism to recover their funds after the chain has exited the Agglayer ecosystem.

`fetchNewWrappedTokenEvents()` in `step_0.go` scans `NewWrappedToken` events from genesis to the target block using a concurrent worker pool. When `runWorkerPool()` returns an error — due to an RPC failure, timeout, or rate-limit on any block range — the function logs a warning and returns whatever partial event set was collected:

```
err := runWorkerPool(
    ctx, jobs, concurrency,
    func(j blockRangeJob) ([]wrappedTokenEvent, error) {
        return fetchWrappedTokenEventsInRange(ctx, cfg.L2RPCURL, cfg.L2BridgeAddress, j.from, j.to)
    },
    func(events []wrappedTokenEvent) {
        allEvents = append(allEvents, events...)
    },
    "NewWrappedToken",
)
if err != nil {
    log.Warnf("Some NewWrappedToken queries failed: %v", err) // @audit error is swallowed — not returned
}

return allEvents // @audit partial list returned as if scan completed successfully
```

Because the function signature is `func fetchNewWrappedTokenEvents(...) []wrappedTokenEvent`, the caller in `RunStep0()` has no error value to inspect:

```
events := fetchNewWrappedTokenEvents(ctx, cfg, blockNum) // @audit no error return to check
```

The pipeline therefore proceeds unconditionally. `fetchTotalSupplies()` queries `totalSupply()` only for tokens in the partial list, so missing tokens receive no LBT entry. Downstream steps C and D derive SC-locked exit values exclusively from the LBT, meaning those tokens will produce no `BridgeExit` entries in the certificate.

The asymmetry with the companion function `fetchSetSovereignTokenEvents()` — which correctly propagates errors via `return nil, err` — confirms this is an unintentional omission rather than a deliberate design choice.

Under the default configuration (`useAgglayerAdminToStepFCheck=true`), Step F performs a three-way comparison between the LBT, the certificate, and the agglayer's reported token balances. Because the agglayer would report a non-zero balance for a token absent from the LBT while the certificate would carry a zero amount, this mismatch would cause Step F to abort the pipeline. This default behaviour provides a meaningful safety net. However, when `useAgglayerAdminToStepFCheck=false` and `ignoreBalanceMismatch=true` are both set, Step F compares only the LBT against the certificate sum; since both are zero for the missing token, no mismatch is detected, and the pipeline completes silently with an incomplete certificate.

This issue has a high impact because the exit certificate is a one-shot, irreversible operation. Any token whose supply is absent from the certificate cannot be recovered by token holders after the chain exits. The impact is direct and

permanent.

This issue has a low likelihood because triggering the defect requires an RPC failure specifically during the `NewWrappedToken` event scan in Step 0. Under normal operating conditions with a stable RPC endpoint, the full event list will be fetched without error. Additionally, the default Step F agglayer check provides a partial safety net that would abort the pipeline before an incomplete certificate is finalised, provided the agglayer admin endpoint is available and the non-default bypass configuration is not in use.

Recommendations

Change `fetchNewWrappedTokenEvents()` to return `([]wrappedTokenEvent, error)` and propagate the `runWorkerPool()` error to the caller. Update `RunStep0()` to check and return the error, aborting the pipeline on failure:

```
func fetchNewWrappedTokenEvents(ctx context.Context, cfg *Config, toBlock uint64) ([]wrappedTokenEvent, error) {
    // ... existing setup ...
    if err := runWorkerPool(...); err != nil {
        return nil, fmt.Errorf("fetch NewWrappedToken events: %w", err)
    }
    return allEvents, nil
}
```

This aligns `fetchNewWrappedTokenEvents()` with the pattern already used by `fetchSetSovereignTokenEvents()`, making the two functions structurally consistent and eliminating the silent failure path.

Resolution

The issue has been fixed in [PR#1721](#). The error is now returned and propagated.

AET-07	Dropped <code>json.Unmarshal()</code> Error In <code>toAgglayerCertificate()</code> Silently Produces An Empty Exit Certificate		
Assets	tools/exit_certificate/run.go		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

`toAgglayerCertificate()` silently discards `json.Unmarshal()` errors when deserialising the `bridge_exits` and `imported_bridge_exits` fields from an on-disk certificate file. When this deserialisation fails, the certificate's `BridgeExits` and `ImportedBridgeExits` fields are left as `nil`, and execution continues without any error or warning. This empty certificate then flows through Steps E, F, G2, and I unchanged, and may ultimately be signed and submitted to the agglayer with zero bridge exit entries, resulting in a migration that moves no funds.

`toAgglayerCertificate()` in `run.go` is the single deserialisation point used by every individual step re-run mode (`runSingleE()`, `runSingleF()`, `runSingleG2()`, `runSingleI()`). The `certificateJSON` struct stores `BridgeExits` and `ImportedBridges` as `json.RawMessage`, which means the outer `loadJSON()` call accepts any syntactically valid JSON for those fields without validating their inner structure. The inner decode — the only place a schema or type error can be caught — is performed inside `toAgglayerCertificate()`, where both errors are unconditionally discarded.

```
// run.go - toAgglayerCertificate
func (c *certificateJSON) toAgglayerCertificate() *agglayertypes.Certificate {
    cert := &agglayertypes.Certificate{
        NetworkID:      c.NetworkID,
        Height:         c.Height,
        PrevLocalExitRoot: c.PrevLocalExitRoot,
        NewLocalExitRoot: c.NewLocalExitRoot,
    }
    if len(c.BridgeExits) > 0 {
        _ = json.Unmarshal(c.BridgeExits, &cert.BridgeExits) // @audit: error dropped; on failure cert.BridgeExits stays nil
    }
    if len(c.ImportedBridges) > 0 {
        _ = json.Unmarshal(c.ImportedBridges, &cert.ImportedBridgeExits) // @audit: error dropped; on failure
        ↪ cert.ImportedBridgeExits stays nil
    }
    return cert
}
```

The dropped error means the caller receives a structurally valid `*agglayertypes.Certificate` pointer with a `nil` slice, which is indistinguishable at the call site from a certificate that legitimately contains no exits. Downstream steps treat this condition as normal. In particular, Step G2 has an explicit fast-path that short-circuits to the canonical empty local exit root when `len(certificate.BridgeExits) == 0`, producing the wrong `NewLocalExitRoot` without any diagnostic output.

The full in-memory `runAll()` pipeline is not affected because it never reads certificate data from disk through this function. The vulnerability is confined to the four single-step re-run entry points.

A concrete operational failure path is as follows. An operator runs the full pipeline with one version of the tool, producing `step-d-exit-certificate.json`. The tool is subsequently updated and the `agglayertypes.BridgeExit` JSON schema changes (the type uses custom `UnmarshalJSON()` logic and is subject to breaking changes). When the operator re-runs `-step g2` to recompute the local exit root, `toAgglayerCertificate()` calls the inner `unmarshal` against a file written in the old schema format. The `unmarshal` fails, the error is dropped, and `BridgeExits` is `nil`. Step G2 observes `len(certificate.BridgeExits) == 0`, skips the shadow-fork replay entirely, and returns `bridgesynctypes.EmptyLER`. The re-ordered certificate is saved with zero exits and an incorrect local exit root. Steps I and SIGN consume this file without

raising any alarm, producing a signed certificate with no bridge exits. If submitted, the settled certificate migrates no funds.

This issue has a high impact because a zero-exit certificate, once settled by the agglayer, leaves all L2 funds locked in the bridge contract with no straightforward recovery path. The likelihood is low because the trigger requires either a schema incompatibility between tool versions or deliberate corruption of on-disk intermediate files; normal single-version usage is unaffected.

Recommendations

Change `toAgglayerCertificate()` to return an error so that JSON decode failures are surfaced to the caller rather than silently producing a nil slice. Update all four call sites in `runSingleE()`, `runSingleF()`, `runSingleG2()`, and `runSingleI()` to propagate the error and halt execution.

This ensures any schema incompatibility or file corruption is caught before the pipeline proceeds with an empty certificate, giving the operator a clear signal to investigate the intermediate files before continuing.

Resolution

The issue has been fixed in [PR#1716](#). The error is now propagated.

AET-08	Missing Zero-Address Validation On Bridge Contract Binding Produces Silent Incorrect Exit Root		
Assets	tools/exit_certificate/bridgescertlite/syncer.go		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The `dialBridge()` function binds the bridge contract without verifying that `cfg.BridgeAddr` is non-zero. If the configured bridge address resolves to the zero address, the syncer silently scans the wrong address, finds no `BridgeEvent` logs, and `BuildTree()` produces the zero hash as the Local Exit Root (LER) — an entirely incorrect result returned without any error.

The binding constructor `agglayerbridge.NewAgglayerbridge()` does not validate the supplied address, so a zero address is accepted as though it were valid:

```
func dialBridge(
    ctx context.Context, cfg Config,
) (*ethclient.Client, *agglayerbridge.Agglayerbridge, error) {
    if cfg.RPCURL == "" {
        return nil, nil, nil
    }
    client, err := ethclient.DialContext(ctx, cfg.RPCURL)
    if err != nil {
        return nil, nil, fmt.Errorf("dial RPC %s: %w", cfg.RPCURL, err)
    }
    // @audit cfg.BridgeAddr is never checked for the zero address before binding
    contract, err := agglayerbridge.NewAgglayerbridge(cfg.BridgeAddr, client)
    if err != nil {
        client.Close()
        return nil, nil, fmt.Errorf("instantiate bridge contract binding: %w", err)
    }
    return client, contract, nil
}
```

This weakness is reachable from configuration. The upstream `validateRawConfig()` in `config.go` checks only that `l2BridgeAddress` is a non-empty string; it never validates the hex format or rejects the zero address. The raw string is then converted with `common.HexToAddress()`, which silently returns the zero address for any malformed input. A typo, an empty-but-present field, or an address loaded from the wrong config section therefore flows through to `dialBridge()` as `0x00...00`. Once bound, every `FilterLogs()` call in `fetchWindow()` queries the zero address, returns zero logs, and the reconstructed exit tree is empty — yielding a zero LER that is then written into the exit certificate.

The impact is rated high because the LER is a security-critical output: a silently incorrect (zero) exit root could be assembled into a certificate that misrepresents the chain's bridge state, and the failure is completely silent with no error or warning to alert the operator. The likelihood is rated low because it requires a misconfigured or malformed bridge address, and a correctly configured run with a valid address is not affected.

Recommendations

Reject the zero address before binding the contract. Add an explicit guard in `dialBridge()`:

```
if cfg.BridgeAddr == (common.Address{}) {  
    return nil, nil, errors.New("bridge address is the zero address; set a valid Config.BridgeAddr")  
}  
contract, err := agglayerbridge.NewAgglayerbridge(cfg.BridgeAddr, client)
```

Additionally, strengthen `validateRawConfig()` to validate the bridge address at the configuration boundary, mirroring the existing `exitAddress` checks: use `common.IsHexAddress()` to reject malformed input and explicitly reject the zero address with a descriptive error. Applying the check at both layers ensures the misconfiguration is caught early regardless of how the syncer is invoked.

Resolution

The comments on [PR#1701](#) indicate that the zero address may hold assets; dropping it left the value uncovered and the certificate unbalanced against the LBT.

AET-09	BridgeSyncerLite.GetBridges() Unbounded Full-Table Load Causes Out-Of-Memory Crash At Scale		
Assets	tools/exit_certificate/bridgesyncerlite/syncer.go		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

`BridgeSyncerLite.GetBridges()` loads the entire bridge table into process memory via an unbounded `SELECT *` with no `LIMIT`, no pagination, and no context cancellation. On a production-scale L2 chain with millions of historical deposits, this exhausts available RAM and crashes the exit certificate pipeline at Step G2, preventing the operator from producing the exit certificate.

The SQL query constant contains no bound:

```
// @audit no LIMIT - returns every row in the bridge table
queryAllBridges = "SELECT * FROM " + bridgeTableName + " ORDER BY deposit_count ASC"
```

`BridgeSyncerLite.GetBridges()` passes this to `meddler.QueryAll()`, which uses `db.Query()` (not `db.QueryContext()`), so the `ctx` parameter is never propagated and cancellation has no effect. After `meddler.ScanAll()` appends all rows into `ptrs []*BridgeLeaf`, a second full-size value copy is constructed — both slices remain alive simultaneously:

```
func (s *BridgeSyncerLite) GetBridges(ctx context.Context) ([]BridgeLeaf, error) {
    // ... snip ...
    var ptrs []*BridgeLeaf
    // @audit unbounded load - no LIMIT, no context cancellation
    if err := meddler.QueryAll(s.db, &ptrs, queryAllBridges); err != nil {
        return nil, fmt.Errorf("query bridges: %w", err)
    }
    // @audit second full-size allocation while ptrs also lives
    bridges := make([]BridgeLeaf, len(ptrs))
    for i, p := range ptrs {
        bridges[i] = *p
    }
    return bridges, nil
}
```

`BridgeSyncerLite.BuildTree()` — the sole caller, invoked by Step G2 (`step_g_events.go`) — holds the returned slice resident throughout its leaf-insertion loop. Each `BridgeLeaf` occupies approximately 200+ bytes (fixed fields plus heap-allocated `*big.Int` and variable-length metadata). At 1 million deposits peak memory reaches approximately 440 MB; at 5 million deposits it approaches 2.2 GB.

Adjacent functions in the same file (`BridgeSyncerLite.CountBridges()` and `BridgeSyncerLite.NextDepositCount()`) use aggregate SQL queries to remain $O(1)$, demonstrating awareness of the scalability concern. `GetBridges()` was not similarly protected.

No adversarial action is required — the condition arises during normal operation on any production chain with sufficient bridge history. Recovery requires provisioning additional RAM and re-running Step G2 only (the Step G1 database is preserved intact across runs).

This issue has a medium impact because the pipeline cannot produce a valid exit certificate until the OOM is resolved, though no funds are at risk. This issue has a medium likelihood because any production L2 active for months to years will accumulate the deposit counts needed to trigger the crash.

Recommendations

Replace the unbounded full-table load in `BridgeSyncerLite.BuildTree()` with a streaming row iterator that uses `db.QueryContext()` (propagating cancellation) and feeds leaves directly into `exitTree.PutLeaf()` one at a time, reducing memory from $O(N)$ to $O(1)$. If `BridgeSyncerLite.GetBridges()` must be retained for other callers, add a documentation comment warning it is unsafe for mainnet-scale histories.

Resolution

The issue was acknowledged by the development team.

AET-10	Hardcoded <code>dest_net=1</code> In ZkeVM Bridge Service Query Bypasses Configured L2 Network ID		
Assets	tools/exit_certificate/step_e.go		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

`fetchZkevmPendingBridges()` hardcodes `dest_net=1` in the HTTP query string when fetching pending bridges from the zkevm bridge service, instead of using the configured `l2NetworkId`. For any L2 chain whose network ID is not 1, the function queries deposits targeting a different chain entirely, causing Step E's bridge service cross-check to silently pass or spuriously fail against the wrong deposit set.

Step E includes an optional safety mechanism that, when `options.bridgeServiceURL` is configured, cross-checks the unclaimed deposits discovered by the local L1 scan against those reported by a configured bridge service. The check is intended to give the operator confidence that the two data sources agree before the certificate is finalised. For the zkevm bridge service type, this cross-check is performed by `fetchZkevmPendingBridges()`, which constructs the query URL as follows:

```
// step_e.go - fetchZkevmPendingBridges
reqURL := fmt.Sprintf("%s/pending-bridges?dest_net=1&leaf_type=%d&limit=%d&offset=%d",
    // @audit: dest_net=1 is a hardcoded literal; should use the configured l2NetworkId
    baseURL, leafType, bridgeSvcPageSize, offset)
```

The function signature accepts no `networkID` parameter and has no access to the configuration:

```
func fetchZkevmPendingBridges(ctx context.Context, baseURL string, leafType uint32) (map[uint32]struct{}, error) {
```

The call site in `checkBridgeServicePendingBridges()` does not pass `cfg.L2NetworkID`:

```
svcCounts, fetchErr = fetchZkevmPendingBridges(ctx, baseURL, leafType) // cfg.L2NetworkID not threaded through
```

The aggkit bridge service path demonstrates the correct approach — it correctly filters by the configured network ID during result processing:

```
// step_e.go - fetchAggkitPendingBridges
for _, b := range result.Bridges {
    if b.DestinationNetwork == cfg.L2NetworkID { // @audit-ok: correctly uses configured network ID
        matching = append(matching, b)
    }
}
```

In the zkevm path, there is no equivalent filter. The `zkevmDeposit` struct does carry a `DestNet` field, but it is never read after unmarshalling. All deposit counts returned by the service are inserted into `svcCounts` unconditionally, regardless of which network they target.

The downstream comparison in `reportPendingDiscrepancies()` then compares `svcCounts` — which contains deposit counts for network 1 — against `unclaimedAssets`, which is correctly filtered to `cfg.L2NetworkID` from the L1 scan. When the configured network ID is not 1, these two sets reflect deposits on different networks and will not correspond. The result is one of two failure modes: a spurious discrepancy error that blocks the pipeline and forces the operator to disable the cross-check to proceed; or a false pass, where deposit counts from network 1 coincidentally match those from the real L2, giving the operator unwarranted confidence that unclaimed deposits have been correctly identified.

The false-pass scenario is the more dangerous of the two outcomes. Actual unclaimed deposits targeting the correct L2 chain may be absent from the bridge service's view for network 1, but the cross-check reports no discrepancy because

it was comparing against the wrong network. The operator may then submit a certificate missing `bridge_exits` for those unclaimed deposits.

The existing test `TestFetchZkevmPendingBridges()` does not verify that the query URL contains the correct `dest_net` value — the mock server ignores query parameters entirely — so the bug is not caught by the test suite.

This issue has a medium impact. The bridge service cross-check is an optional safety guard; the primary certificate generation from the L1 scan is unaffected. However, in the false-pass scenario, the operator proceeds with a potentially incomplete certificate without realising the verification layer has been silently comparing the wrong data, which undermines the purpose of the cross-check. This issue has a medium likelihood. The bug only manifests when `options.bridgeServiceType` is set to "zkevm" and `l2NetworkId` is not 1. The default `l2NetworkId` of 1 inadvertently protects the majority of deployments, but any operator of a non-primary L2 chain who configures the zkevm bridge service type will encounter this bug.

Recommendations

Pass the configured L2 network ID into `fetchZkevmPendingBridges()` and substitute it into the query string. Update the function signature to accept the network ID as a parameter:

```
func fetchZkevmPendingBridges(ctx context.Context, baseURL string, leafType uint32, l2NetworkID uint32) (map[uint32]struct{},
    ↪ error) {
    // ...
    reqURL := fmt.Sprintf("%s/pending-bridges?dest_net=%d&leaf_type=%d&limit=%d&offset=%d",
        baseURL, l2NetworkID, leafType, bridgeSvcPageSize, offset)
    // ...
}
```

Update the call site in `checkBridgeServicePendingBridges()` to pass `cfg.L2NetworkID`:

```
svcCounts, fetchErr = fetchZkevmPendingBridges(ctx, baseURL, leafType, cfg.L2NetworkID)
```

Update `TestFetchZkevmPendingBridges()` to assert that the HTTP request received by the mock server contains the correct `dest_net` query parameter value.

Resolution

The issue has been fixed in [PR#1683](#). The `dest_net` parameter is now an input.

AET-11	Unsettled L2 Bridge Exits Can Block The Exit Pipeline		
Assets	tools/exit_certificate/step_g1.go tools/exit_certificate/step_g2.go tools/exit_certificate/step_h.go tools/exit_certificate/README.md		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Step H requires the L2 bridge's local exit root at the frozen target block to equal the agglayer's last settled local exit root. A normal L2 bridge withdrawal made after the last settled certificate but before the sequencer is halted changes the L2 local exit root, but the tool does not include that already-emitted bridge exit in the certificate. The pipeline then reaches Step H and aborts with a root mismatch.

The README lists the operational precondition as stopping the sequencer before running the tool:

```
Halt the sequencer before running the tool so that no new bridges (or other state changes) are produced while the certificate is
↪ being built.
```

That precondition prevents new bridge events during certificate generation, but it does not prevent ordinary L2 bridge exits that were created before the halt and after the last agglayer settlement.

Step G1 syncs the L2 bridge history through the frozen `targetBlock`:

```
if err := syncLiteToBlock(ctx, cfg, targetBlock); err != nil {
    return nil, fmt.Errorf("lite-sync L2 bridges up to block %d: %w", targetBlock, err)
}
```

`syncLiteToBlock()` includes all bridge events in `[0..targetBlock]`:

```
if err := syncer.Sync(ctx, 0, targetBlock); err != nil {
    return err
}
```

Step G2 then records the bridge contract's local exit root at the forked target block as `InitialLocalExitRoot`:

```
initialLER, err := backend.LocalExitRoot(ctx, "latest")
if err != nil {
    return common.Hash{}, common.Hash{}, nil, fmt.Errorf("read initial local exit root: %w", err)
}
```

Step H fetches the agglayer's last settled local exit root:

```
if info.SettledLER != nil {
    prevLER = *info.SettledLER
}
```

It then requires the target-block L2 root to equal that settled agglayer root:

```
if gResult.InitialLocalExitRoot != prevLER {
    return nil, fmt.Errorf(
        "LocalExitRoot mismatch: Step G started from %s (read from bridgeContract) but agglayer last settled %s ...",
        gResult.InitialLocalExitRoot.Hex(), prevLER.Hex(),
    )
}
```


The check is internally consistent: the final exit certificate is only valid if it starts from the agglayer's settled LER. The issue is that the documented operational requirements do not ensure that condition. Permissionless L2 bridge exits can be created before the halt, and those leaves advance the L2 bridge root even when no certificate carrying them has settled on the agglayer.

A concrete failure path is:

1. The agglayer's last settled local exit root for the L2 is R_0 .
2. Before the planned shutdown, a user performs a normal L2-to-L1 bridge withdrawal.
3. The L2 bridge emits a `BridgeEvent` and advances its local exit root from R_0 to R_1 .
4. No agglayer certificate settling R_1 is submitted or finalized before the operator halts the sequencer.
5. The operator runs the exit tool at target block T , which includes the user's L2 bridge event.
6. Step G2 reads `InitialLocalExitRoot =  $R_1$` from the frozen L2 bridge.
7. Step H reads `settled_ler =  $R_0$` from the agglayer.
8. Step H aborts because $R_1 \neq R_0$, so the final certificate is not produced.

The impact is a permissionless liveness failure for the exit process. A normal user action that is valid while the sequencer is still running can make the documented shutdown procedure fail after the expensive state scan and replay phases. The operator must either wait for the outstanding L2 bridge exit to settle in an agglayer certificate before taking the exit snapshot, or restart the chain and choose a new snapshot. If users can keep bridging before each halt, they can repeatedly force the operator back into the same recovery loop.

The likelihood is medium ~ L2 bridge withdrawals are normal and permissionless, and the vulnerable window is the period between the last settled agglayer certificate and the final sequencer halt. The issue does not require malformed tokens, RPC failure, malicious infrastructure, or privileged access. It only requires an ordinary bridge event that has not yet been reflected in the agglayer's `settled_ler`.

Recommendations

Add an explicit preflight requirement that the L2 bridge root at the intended target block equals the agglayer's `settled_ler` before the full exit pipeline proceeds. Run this check before the expensive balance scan and Step G replay work, and document that all L2 bridge exits up to the target block must be settled before the final snapshot is taken.

At minimum:

```
l2LER, err := readLocalExitRoot(ctx, cfg.L2RPCURL, cfg.L2BridgeAddress, toBlockTag(targetBlock))
if err != nil {
    return nil, fmt.Errorf("read L2 local exit root at target block: %w", err)
}
if l2LER != settledLER {
    return nil, fmt.Errorf("target block has unsettled L2 bridge exits: L2 LER %s, agglayer settled LER %s",
        l2LER.Hex(), settledLER.Hex())
}
```

If the tool is expected to handle unsettled L2 bridge exits, it should instead import the missing bridge leaves into the final certificate explicitly and compute the new root from the agglayer's settled LER, not from the already-advanced L2 target root.

Resolution

The issue has been fixed in [PR#1724](#). A new option in `cfg.Options.IgnoreLERMismatch` in `step_check.go` allows for an early exit if the L2 bridge root at the target block does not match the agglayer's settled LER. The tool will now fail fast with a clear error message, preventing wasted computation in the balance scan and replay steps.

AET-12	uint64 Addition Overflow In decodeABIString Bypasses Bounds Guard And Crashes The Process		
Assets	tools/exit_certificate/step_e.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

A `uint64` integer overflow in `decodeABIString()` allows a maliciously crafted L1 ERC-20 token whose `name()` function returns an oversized ABI length field to bypass the function's bounds guard and trigger a process-fatal panic, permanently preventing the exit certificate tool from generating a certificate for any chain where such a token has an unclaimed L1-to-L2 deposit.

`decodeABIString()` in `step_e.go` decodes the ABI-encoded return value of a `name()` call. It reads the 32-byte length word from the response using `big.Int.SetBytes(...).Uint64()`, then guards against out-of-bounds access with the check `64+strLen > uint64(len(data))`. Both operations are performed in `uint64` arithmetic:

```
func decodeABIString(data []byte) string {
    if len(data) < twoABIWords {
        return ""
    }
    strLen := new(big.Int).SetBytes(data[32:64]).Uint64() // @audit Uint64() silently truncates values > MaxUint64
    if 64+strLen > uint64(len(data)) {                     // @audit 64+strLen overflows when strLen >= MaxUint64-63
        return ""                                          // the guard incorrectly passes
    }
    return string(data[64 : 64+strLen])                  // @audit 64+strLen wraps to a value < 64; data[64:63] panics
}
```

`big.Int.Uint64()` returns only the low 64 bits of the integer, silently truncating any value whose encoding exceeds `MaxUint64`. When the ABI length word encodes `MaxUint64` (for example, 32 bytes of `0xFF`), `strLen` is assigned `18446744073709551615`. The addition `64 + MaxUint64` overflows `uint64` and wraps to `63`. The bounds check then evaluates `63 > uint64(len(data))`: since `len(data) >= 64` was already confirmed, this comparison is false and the guard incorrectly passes. The subsequent slice expression `data[64 : 63]` presents a lower bound greater than its upper bound and the Go runtime panics with `runtime error: slice bounds out of range [64:63]`.

No `recover()` exists anywhere in the `tools/exit_certificate` package, so the panic is process-fatal and no certificate output is written.

The panic is reached via the following unconditional call path:

```
RunStepE
-> logUnclaimedAssetSummary // called at line 84, before any IgnoreUnclaimed check
-> fetchTokenInfo
-> fetchTokenName
-> decodeABIString // PANIC here
```

`logUnclaimedAssetSummary()` is invoked at line [84], before the `options.ignoreUnclaimed` branch at line [94], so the `ignoreUnclaimed` configuration option does not prevent the crash.

The attack is fully permissionless. An adversary deploys a malicious ERC-20 on L1 whose `name()` function returns a 64-byte ABI response with an all-`0xFF` length word (`0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF`). The deployer then bridges a trivially small amount of this token to the target L2 using the standard permissionless `PolygonZkEVMBridgeV2` bridge and leaves the deposit unclaimed. When the chain operator runs Step E, the tool finds the unclaimed deposit, calls `logUnclaimedAssetSummary()`, fetches `name()` from the malicious contract, and crashes before any output is written. Because the malicious deposit remains in the L1 bridge event history, every subsequent run

will trigger the same crash until the operator patches the tool or filters the offending deposit count.

This issue has a medium impact because, while the crash prevents certificate generation, it occurs in a logging-only code path (`logUnclaimedAssetSummary`) that fetches cosmetic token names — the token name is not included in the certificate itself. The panic is recoverable: the operator can patch the tool, filter the offending deposit count, or skip the logging call without affecting certificate correctness. The denial is temporary rather than permanent from an operational perspective. This issue has a low likelihood because the attack requires the adversary to deploy and bridge a crafted token on L1, which is permissionless and inexpensive, but presupposes advance knowledge that the target chain intends to exit the Agglayer.

Recommendations

Replace the `Uint64()` call with an overflow-safe check using `big.Int.IsUint64()` before converting, and eliminate the overflow-prone addition in the bounds guard:

```
func decodeABIString(data []byte) string {
    if len(data) < twoABIWords {
        return ""
    }
    strLenBig := new(big.Int).SetBytes(data[32:64])
    if !strLenBig.IsUint64() || strLenBig.Uint64() > uint64(len(data)-64) {
        return ""
    }
    strLen := strLenBig.Uint64()
    return string(data[64 : 64+strLen])
}
```

The check `strLenBig.Uint64() > uint64(len(data)-64)` is safe because `len(data) >= 64` is already confirmed, so `len(data)-64 >= 0`. This form avoids the `uint64` addition entirely, making the overflow impossible.

Resolution

The issue has been fixed in [PR#1684](#). Checks were added to ensure that the length word is a valid `uint64` and that the decoded length does not exceed the bounds of the data slice.

AET-13	checkGenesisBalances() Does Not Query Contract Balances At Genesis Block		
Assets	tools/exit_certificate/step_b.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The genesis preload guard in `checkGenesisBalances()` only queries EOA addresses at block 0, leaving contract addresses entirely unchecked at genesis.

A chain where ETH was pre-loaded exclusively into contract addresses at genesis will pass the guard undetected, producing an exit certificate with over-inflated SC-locked value not backed by actual bridge inflows. The function's own comment states it "fetches ETH balances at block 0 for EOAs and contracts," but the implementation queries contract addresses at `blockTag` (the target block) rather than block 0, using that result solely in a diagnostic log. The guard condition `if len(genesiBalances) == 0 { return nil }` depends solely on the EOA genesis query:

```
scBalances, err := fetchETHBalances(ctx, rpcURL, contractAddrs, blockTag, batchSize, concurrency) // target block, not genesis
genesisBalances, err := fetchETHBalances(ctx, rpcURL, eoaAddrs, toBlockTag(0), batchSize, concurrency)
if len(genesiBalances) == 0 {
    return nil // contract genesis preloads pass undetected
}
```

This issue has a medium impact because it silently defeats the genesis guard — the operator's only protection against this class of deployment error — and a low likelihood because most production PP chains do not pre-load ETH into contracts at genesis.

Recommendations

Extend `checkGenesisBalances()` to also query contract addresses at block 0 and include those results in the guard condition. Add a corresponding test case that verifies the pipeline aborts when a contract has a non-zero balance at genesis.

Resolution

The issue was acknowledged by the development team.

AET-14	Duplicate bridgeAsset() Transaction On Post-Delivery Connection Reset Aborts Step G2		
Assets	tools/exit_certificate/step_g2.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

A transport-layer timeout or connection reset after an `eth_sendTransaction` request has been delivered to Anvil causes `sendAnvilTransaction()` to retry, submitting a duplicate `bridgeAsset()` transaction that inserts an extra leaf into the onchain exit tree and aborts Step G2 with a root-mismatch error.

`sendAnvilTransaction()` constructs a transaction without an explicit `nonce` field and retries on any error matched by `isTransientForkError()`, which classifies "connection" and "eof" markers as retryable. A code comment asserts "the send never landed, so retrying is safe," but when the HTTP POST body has already been received by Anvil, the transaction is in the mempool before the response is sent. On retry, Anvil auto-assigns the next sequential nonce, producing a distinct transaction. Both mine, each emitting a `BridgeEvent` leaf. The offchain lite tree, built from exactly N certificate exits, cannot match the contract's `getRoot()` (N+1 leaves), so the verification aborts. Layered retries (5 outer × 3 inner = up to 15 HTTP POSTs per exit) amplify the window. The correct pattern already exists in `retryDeferredExit()`, which re-polls the original transaction hash before deciding to re-send.

```
// step_g2.go - sendAnvilTransaction()
for attempt := 1; ; attempt++ {
    result, err = singleRPC(ctx, anvilURL, "eth_sendTransaction", [any]{tx}, defaultRetries)
    if err == nil {
        break
    }
    // @audit assumes "the send never landed" - incorrect for post-delivery errors
    if !isTransientForkError(err) || attempt >= forkRetryAttempts {
        return common.Hash{}, err
    }
    // ... retries without checking if the original tx already landed ...
}
```

This issue has a medium impact as the consequence is an abort of Step G2 with a clear root-mismatch error, forcing a full pipeline restart (hours for large mainnet replays), though no incorrect certificate is silently produced. This issue has a low likelihood as it requires a remote fork backend under high concurrent load where the HTTP client timeout (120 s) expires while Anvil blocks on a cold-state fetch, which is a supported but explicitly discouraged configuration.

Recommendations

Before retrying in `sendAnvilTransaction()`, query `eth_getTransactionByHash` to check whether the original transaction already landed in the mempool, mirroring the re-poll-before-resend pattern in `retryDeferredExit()`. If the transaction is present, skip the re-send and use the existing hash.

Resolution

The issue was acknowledged by the development team.

AET-15	Exit Tree Built From Non-Transactional Bridge Snapshot		
Assets	aggkit/tools/exit_certificate/bridgesyncerlite/syncer.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

`BuildTree()` reads the full set of bridge leaves with `s.GetBridges(ctx)` before it opens the database transaction that writes the exit tree, so the tree is assembled from a snapshot that is not guaranteed to match the transactional state of the database. If any write occurred between the read and the transaction, the resulting local exit root would be computed over stale data while the database holds a different set of bridges.

In `BuildTree()` the leaves are loaded outside the transaction:

```
bridges, err := s.GetBridges(ctx) // @audit read happens before the transaction is opened
if err != nil {
    return common.Hash{}, err
}
if len(bridges) == 0 {
    s.log.Info("no bridges stored; exit tree is empty")
    return common.Hash{}, nil
}

tx, err := db.NewTx(ctx, s.db) // @audit transaction opened only after the read above
if err != nil {
    return common.Hash{}, fmt.Errorf("begin transaction: %w", err)
}
```

`GetBridges()` runs its own standalone query against `s.db` rather than against `tx`. Consequently there is a read-then-write gap: if `StoreBridges()` were invoked concurrently between the `GetBridges()` call and `db.NewTx()`, the newly stored bridges would be committed to the database while the tree is built only from the earlier snapshot. The exit tree would then be missing those leaves, producing a local exit root that does not reflect the bridges actually persisted. Because the local exit root is the security-critical value the tool exists to compute, any silent divergence between the persisted bridges and the tree built over them undermines the correctness guarantee of the whole pipeline.

The likelihood is assessed as low because the tool is intended to run as a single sequential pipeline (each step runs to completion before the next), `BridgeSyncerLite` is not designed for concurrent use, and there is no code path in the current tool that calls `StoreBridges()` while `BuildTree()` is executing. The impact is assessed as medium because, should the read and write ever interleave, the divergence is silent — no error is raised — and a wrong local exit root could be produced and acted upon, but realising this requires concurrency that the tool does not currently exhibit.

Recommendations

Perform the read inside the same transaction that builds the tree, so the snapshot used to assemble the tree and the writes are atomic and consistent. Open the transaction first, then query the bridges through that transaction handle (for example, by adding a variant of `GetBridges()` that accepts the `tx/db.Querier` and using it after `db.NewTx()` succeeds). This guarantees the tree is always built from exactly the state captured by the transaction.

Resolution

The issue was acknowledged by the development team.

AET-16	Non-Deterministic Last-Write-Wins In <code>overrideMap</code> Silently Records Wrong Sovereign Token Address		
Assets	tools/exit_certificate/step_0.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

When the same origin token has been remapped more than once via `SetSovereignTokenAddress`, the `overrideMap` in `step_0.applySovereignTokenOverrides()` records whichever sovereign address arrived last from the concurrent worker pool rather than the chronologically latest onchain event. The function fetches all override events using `runWorkerPool()`, which appends results in goroutine-completion order, not block-number order. The collected overrides then build `overrideMap` with unconditional last-write-wins assignment per origin key:

```
overrideMap := make(map[originKey]common.Address, len(overrides))
for _, ov := range overrides {
    if ov.SovereignAddr != (common.Address{}) {
        overrideMap[originKey{ov.OriginNetwork, ov.OriginTokenAddress}] = ov.SovereignAddr
    }
}
```

If the batch containing the earlier event completes after the batch containing the later event, the stale sovereign address overwrites the current one. The LBT then records the wrong wrapped token, and `LBTEntriesToWrappedTokens()` in `config.go` never propagates intermediate sovereign addresses to Steps B through D, so holders of the actual live token are silently excluded from the exit certificate. This issue has a medium impact and low likelihood because it requires a chain that called `SetSovereignTokenAddress` more than once for the same origin token — an uncommon privileged operation — and the race only manifests when the relevant batches complete in reverse chronological order.

Recommendations

Add a `BlockNumber` field to `sovereignTokenOverride` (from the log's `blockNumber`), sort `allOverrides` by block number in ascending order after the worker pool completes, and update `step_0.applySovereignTokenOverrides()` to record all intermediate sovereign addresses in `LegacyAddrs` so downstream steps query balances for all historical sovereign addresses.

Resolution

The issue has been fixed in [PR#1711](#). A sorting feature was added.

AET-17	Non-deterministic Token Ordering In <code>buildSingleEOABalance()</code> Produces Irreproducible Offchain LER		
Assets	tools/exit_certificate/step_b.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

Across separate pipeline invocations in offchain G2 mode, nondeterministic map iteration in `step_b.buildSingleEOABalance()` causes the `NewLocalExitRoot` to vary between runs for the same onchain state, potentially causing certificate rejection if steps are re-run independently.

`step_b.buildSingleEOABalance()` ranges over the `tokenBalances` map (`map[common.Address]map[common.Address]*big.Int`), whose iteration order Go randomises per-run by specification. The resulting `entry.Tokens` slice order propagates through `step_d.eoaToExits()` into `Certificate.BridgeExits`, and into the positional deposit-count assignment in `step_g_events.buildLiteTreeFromCertificate()` (`DepositCount: nextDepositCount + uint32(i)`). Since the `BridgeExit` struct carries no explicit deposit count field, both the tool and the agglayer assign deposit counts from array position — different orderings produce different Merkle roots.

```
// step_b.go - buildSingleEOABalance()
for tokenAddr, holders := range tokenBalances { // @audit nondeterministic map iteration
    if bal, ok := holders[tokenAddr]; ok && bal.Sign() > 0 {
        info := tokenLookup[tokenAddr]
        entry.Tokens = append(entry.Tokens, EOATokenBalance{ // @audit Tokens order is random
            WrappedTokenAddress: tokenAddr,
            // ... snip ...
        })
    }
}
```

A single `runAll` execution is internally consistent (same in-memory object flows through the pipeline), and the default shadow-fork mode is immune because `step_g_order.reorderCertificateByDepositCount()` canonicalises exit order after replay. The issue only manifests in offchain mode when the pipeline is split across invocations or re-run, producing a different `step-b-eoa-balances.json` ordering each time.

This issue has a medium impact as it can cause certificate rejection when split-step re-runs produce a mismatched LER, requiring the operator to re-execute the full pipeline. This issue has a low likelihood as it requires the non-default offchain G2 mode (`verifyNewLocalExitRootUsingShadowFork=false`), at least one EOA holding multiple wrapped tokens, and a split-step or re-run workflow rather than the typical single `runAll` invocation.

Recommendations

Sort `entry.Tokens` by `WrappedTokenAddress` before returning from `step_b.buildSingleEOABalance()`. Apply the same deterministic sort in `step_b.buildAccumulated()`. No changes to downstream steps are required once the upstream ordering is stable.

Resolution

The issue has been fixed in [PR#1711](#). A sorting feature was added.

AET-18 Non-Nil Empty LBT Slice Incorrectly Activates Three-Way Comparison			
Assets	tools/exit_certificate/step_f.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

When Step F is re-run in single-step mode and the LBT file contains a JSON `null` value (written when Step 0 found no tokens), `config.LoadLBTEntries()` returns a non-nil empty slice rather than a nil slice. Because `step_f.compareTokenBalances()` uses a nil check (`lbtEntries != nil`) rather than a length check to decide whether to activate the three-way agglayer/LBT/certificate comparison, the three-way path is incorrectly activated with an empty LBT map. Every token lookup in the LBT map returns nil, which is substituted with zero. With `ignoreBalanceMismatch=true`, this sets every token's cap budget to zero and `step_f.capCertificateExits()` drops all bridge exits from the certificate.

`step_f.compareTokenBalances()` determines whether to run a two-way or three-way comparison using the following check:

```
hasLBT := lbtEntries != nil // @audit checks nil-ness, not emptiness
                        // a non-nil empty slice sets hasLBT=true
                        // but lbtMap has zero entries, so all lbtAmt = 0
```

When `hasLBT` is true and `lbtMap` is empty, every key lookup defaults to `new(big.Int)` (zero):

```
if hasLBT {
    lbtAmt := lbtMap[k]
    if lbtAmt == nil {
        lbtAmt = new(big.Int) // @audit defaults to 0 for every key when lbtMap is empty
    }
    check.LBTAmt = lbtAmt.String()
    check.Match = certAmt.Cmp(agglAmt) == 0 && agglAmt.Cmp(lbtAmt) == 0
    if agglAmt.Cmp(lbtAmt) <= 0 {
        check.RemainingBalance = new(big.Int).Set(agglAmt)
    } else {
        check.RemainingBalance = new(big.Int).Set(lbtAmt)
        // @audit lbtAmt=0 → RemainingBalance=0 for every token with agglAmt>0
    }
}
```

The source of the non-nil empty slice is `config.LoadLBTEntries()`. When the LBT file contains the JSON value `null`, `json.Decode` leaves the target `[]rawLBTEnt` variable as nil. `config.LoadLBTEntries()` then calls `make([]LBTEnt, len(nil))`, which produces `make([]LBTEnt, 0)` — a non-nil slice with length zero. This slice is returned without error and assigned to `lbtEntries` in `run.runSingleF()`.

The rest of the codebase consistently uses length-based checks to test whether LBT data is meaningful. `step_f.runStepFOfflineLBT()` uses `if len(lbtEntries) == 0` to skip the offline path. `run.runAllStepC()` also uses `len(lbtEntries) == 0` as its skip guard. `step_f.compareTokenBalances()` is the sole function using a nil check for this decision.

The full code path for the vulnerable scenario is: Step 0 runs on a chain with no wrapped tokens, returning nil for `result.Entries`. `run.saveJSON()` serialises nil as "null" to `step-0-lbt.json`. The operator later re-runs Step F alone (`-step f`). `config.LoadLBTEntries()` reads the file, decodes `null` to a nil slice, calls `make([]LBTEnt, 0)`, and returns a non-nil empty slice. `step_f.RunStepF()` is invoked with `useAgglayerAdminToStepFCheck=true` (the default). `step_f.compareTokenBalances()` sets `hasLBT=true`, builds an empty `lbtMap`, and for every token — all of which

have a real agglayer balance greater than zero — computes `agglAmt > lbtAmt(0)`, setting `RemainingBalance=0`. With `ignoreBalanceMismatch=true`, `step_f.capCertificateExits()` drops every exit, returning an empty `BridgeExits` slice.

This defect is specific to single-step (`run.runSingleF()`) execution. The full pipeline (`run.runAll()`) passes the `nil result.Entries` directly to `step_f.RunStepF()` without a file round-trip, so `lbtEntries` is `nil` at that point and `hasLBT=false`. The two-way agglayer-vs-certificate comparison is used correctly in that path.

This issue has a medium impact. While the bug technically produces a certificate with zero bridge exits, multiple visible safeguards limit the realistic damage. Step F prints explicit "â€” MISMATCH" log lines for every token, followed by "Balance mismatches detected â€” continuing anyway (`ignoreBalanceMismatch=true`)" and "ðŸ”” Capped certificate: N â€” 0 bridge exits". Step G2 subsequently logs "No bridge exits â€” using EmptyLER". These warnings make the anomaly highly visible to an operator reviewing the output. Furthermore, the `SUBMIT` step is not part of `runAll` and must be triggered explicitly with `-step submit`, meaning the operator must independently decide to submit a zeroed certificate after observing these warnings.

This issue has a low likelihood because it requires three conditions to hold simultaneously: a chain where Step 0 produces `nil` LBT entries (no wrapped tokens), Step F being re-run in single-step mode rather than as part of the full pipeline, and `ignoreBalanceMismatch=true` being explicitly set (a non-default configuration flag).

Recommendations

Replace the `nil` check in `step_f.compareTokenBalances()` with a length check, making it consistent with the rest of the codebase:

```
// Before:
hasLBT := lbtEntries != nil

// After:
hasLBT := len(lbtEntries) > 0
```

Also update the corresponding guard at approximately line [112] of `step_f.go` where `lbtEntries != nil` is used for logging:

```
// Before:
if lbtEntries != nil {

// After:
if len(lbtEntries) > 0 {
```

This makes the three-way comparison semantics consistent: an empty LBT is treated identically to a `nil` LBT (no LBT data available), falling back to the two-way agglayer-versus-certificate comparison.

Resolution

The issue was acknowledged by the development team.

AET-19	Null debug_traceTransaction Result Silently Omits Addresses From Exit Certificate		
Assets	tools/exit_certificate/step_a.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

When `debug_traceTransaction` returns a JSON-RPC success response with a null `result` field, `traceOneTransaction()` silently returns an empty address set with no error. The affected transaction is never added to `FailedTraces`, so Step A2 receipt recovery is never triggered. Any address that appeared exclusively in that transaction's pre or post state is permanently omitted from the certificate's address set, and consequently excluded from the final exit certificate.

`traceOneTransaction()` calls `singleRPC()` with `"debug_traceTransaction"` and unmarshals the result into an anonymous struct with `Pre` and `Post` map fields. When the node emits `{"jsonrpc":"2.0","id":1,"result":null}` without an error field, `singleRPC()` returns `json.RawMessage("null")` with a nil error. The subsequent `json.Unmarshal()` call treats null input as a no-op for a non-pointer struct target per the Go JSON specification, leaving `trace.Pre` and `trace.Post` as nil maps. Ranging over nil maps produces zero iterations, so the function returns an empty address slice with a nil error.

```
// step_a.go - traceOneTransaction()
result, err := singleRPC(ctx, rpcURL, "debug_traceTransaction", []any{
    txHash.Hex(),
    map[string]any{
        "tracer": "prestateTracer",
        "tracerConfig": map[string]any{"diffMode": true},
    },
}, defaultRetries)
if err != nil {
    return nil, fmt.Errorf("trace transaction %s: %w", txHash.Hex(), err)
}
// @audit no null-result guard here - contrast with receiptAddresses() which checks:
// if len(result) == 0 || string(result) == "null" { return nil, error }

var trace struct {
    Pre map[string]any `json:"pre"`
    Post map[string]any `json:"post"`
}
if err := json.Unmarshal(result, &trace); err != nil { // @audit succeeds on "null" input
    return nil, fmt.Errorf("unmarshal trace for transaction %s: %w", txHash.Hex(), err)
}

// @audit trace.Pre and trace.Post are nil when result was "null"
// For-range over nil maps iterates zero times - no addresses collected
addrSet := make(map[common.Address]struct{}, len(trace.Pre)+len(trace.Post))
for addr := range trace.Pre {
    addrSet[common.HexToAddress(addr)] = struct{}{}
}
for addr := range trace.Post {
    addrSet[common.HexToAddress(addr)] = struct{}{}
}
return addresses, nil // @audit returns ([], nil) - silent success with zero addresses
```

The worker pool in `traceTransactions()` only appends to `FailedTraces` when `traceErr != nil`. A nil-error zero-address return is invisible to both the `FailedTraces` accumulator and the `IgnoreOnTraceError` safety valve. Step A2 receipt recovery — the designed fallback for trace failures — is never invoked.

The same codebase correctly handles this case in the sibling function `receiptAddresses()`:

```
// step_a.go - receiptAddresses() (Step A2), correct guard
if len(result) == 0 || string(result) == "null" {
    return nil, fmt.Errorf("receipt for %s is null", hash.Hex())
}
```

This issue has a medium impact because Step F's balance verification provides a safety net: by default (`ignoreBalanceMismatch=false`), the pipeline aborts when the certificate's token totals do not match the LBT or agg-layer totals. This means that in the default configuration, a missing address with non-zero balance would cause Step F to halt the pipeline before a certificate is produced. However, when the operator sets `ignoreBalanceMismatch=true` (a legitimate option for handling known small discrepancies), the safety net is bypassed and the omitted address's funds would be excluded from the final certificate. Additionally, the completely silent nature of the failure — no log, no warning, no failed-trace record — means operators have no diagnostic trail pointing to the root cause when Step F does flag a mismatch.

This issue has a low likelihood because it requires the L2 archive node to return null for `debug_traceTransaction` on at least one transaction. This is an edge case for most well-configured archive nodes but is non-trivial for nodes that do not fully support `prestateTracer` with `diffMode` for all transaction types, or for chains with system transactions.

Recommendations

Add the same null-result guard to `traceOneTransaction()` that already exists in `receiptAddresses()`. Insert the check immediately after the `singleRPC()` error check, before the `json.Unmarshal()` call:

```
if len(result) == 0 || string(result) == "null" {
    return nil, fmt.Errorf("trace transaction %s: null result", txHash.Hex())
}
```

By returning an error on a null result, the worker pool will route the transaction to `FailedTraces` (when `IgnoreOnTraceError=true`) or abort the pipeline (when `false`). Step A2 will then recover the addresses from `eth_getTransactionReceipt`, which is the recovery path already designed for trace failures.

Resolution

The issue was acknowledged by the development team.

AET-20	Silent Drop Of BridgeEvent Decode Failures Omits Unclaimed Deposits From Exit Certificate		
Assets	tools/exit_certificate/step_e.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

A failure to ABI-decode an L1 `BridgeEvent` log in `fetchBridgeEventsInRange()` is silently discarded, causing the corresponding deposit to be absent from the exit certificate without operator notification. Given the irreversible nature of a chain exit, a silently omitted deposit means the affected user's funds cannot be recovered through the agglayer once the certificate is settled.

`fetchBridgeEventsInRange()` queries `eth_getLogs` for all `BridgeEvent` logs from the L1 bridge contract, then calls `decodeBridgeEvent()` on each entry. When `decodeBridgeEvent()` returns an error, the loop executes a bare `continue` with no log output, no error accumulation, and no indication to the caller:

```
var deposits []L1Deposit
for _, lg := range logs {
    dep, err := decodeBridgeEvent(lg.Data, lg.BlockNumber, lg.TransactionHash)
    if err != nil {
        continue // @audit decode error silently dropped - deposit omitted
    }
    if dep.DestinationNetwork == l2NetworkID {
        deposits = append(deposits, dep)
    }
}
return deposits, nil // @audit nil error returned even when events were skipped
```

`decodeBridgeEvent()` can error on multiple paths: data shorter than 256 bytes (misconfigured bridge address pointing to a contract with a different data layout), metadata exceeding the one-megabyte limit via `extractMetadata()`, or `safeUint32()/safeUint8()` overflow for field values exceeding their target type's range. In all cases the function returns `nil` error unconditionally and `RunStepE()` treats the truncated `[]L1Deposit` as the complete set. No deposit that fails to decode can appear as an `ImportedBridgeExit` in the final certificate.

The optional bridge-service cross-check (`checkBridgeServicePendingBridges()`) provides partial mitigation when `options.bridgeServiceURL` is configured, as it would detect a discrepancy if the bridge service independently reports the affected deposit. However, this cross-check is not configured in every deployment.

This contrasts with Step A, which accumulates failed traces into a `failedTraces` slice, writes them to `step-a-failed-traces.json` for operator review, and supports an `IgnoreOnTraceError` option to gate whether failures are fatal.

This issue has a medium impact because, while omitting a deposit from an irreversible exit certificate could mean fund loss for the affected user, the realistic trigger conditions are narrow — `decodeBridgeEvent()` only fails on data shorter than 256 bytes, metadata exceeding the one-megabyte limit, or integer overflow in field values. A correctly deployed bridge contract emits well-formed payloads, and the optional bridge-service cross-check (`checkBridgeServicePendingBridges()`) provides partial mitigation when configured by detecting discrepancies against an independent data source. This issue has a low likelihood because the error conditions require a corrupted RPC response, misconfigured bridge address pointing to a different contract, or unusual onchain state that produces malformed event data.

Recommendations

Replace the silent `continue` with error handling that mirrors the Step A `failedTraces` pattern. Introduce a configuration option (e.g. `ignoreOnBridgeEventDecodeError`) that gates whether the step aborts or continues on decode failure. When false (the recommended default), return an error on the first decode failure surfacing the affected transaction hash. When true, log a warning per skipped event, accumulate the failed transaction hashes, and persist them to a dedicated output file for operator review before certificate submission.

Resolution

The issue has been fixed in [PR#1721](#). Error handling was added.

AET-21	Missing Per-Request Timeout On RPC Log Fetch Can Stall Sync Indefinitely		
Assets	aggkit/tools/exit_certificate/bridgesyncerlite/downloader.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

The `fetchWindow()` function calls `FilterLogs()` using the `errgroup` context, which carries no deadline of its own. A slow or unresponsive RPC node can therefore stall a fetch goroutine indefinitely, holding an `errgroup` semaphore slot and ultimately preventing the overall sync from completing.

The bridge log fetch is parallelised in `fetchBridges()`, which splits the block range into windows and dispatches them through an `errgroup` bounded by `s.cfg.Concurrency`. Each goroutine invokes `fetchWindow()`, which in turn issues the RPC call:

```
func (s *BridgeSyncerLite) fetchWindow(ctx context.Context, from, to uint64) ([]BridgeLeaf, error) {
    // @audit ctx is the errgroup context with no deadline; a hung RPC call blocks forever
    logs, err := s.client.FilterLogs(ctx, ethereum.FilterQuery{
        FromBlock: new(big.Int).SetUint64(from),
        ToBlock:   new(big.Int).SetUint64(to),
        Addresses: []common.Address{s.cfg.BridgeAddr},
    })
    if err != nil {
        return nil, err
    }
    return classifyLogs(s.contract, logs, s.cfg.IgnoreUnsupportedL2Events, s.log)
}
```

The `ctx` passed down originates from the caller of `fetchBridges()` via `errgroup.WithContext(ctx)`. Unless that caller attaches a deadline, the context never expires. The `errgroup` only cancels its derived context when a goroutine returns an error, so a call that simply hangs (rather than failing) will never trigger cancellation. Because `g.SetLimit(s.cfg.Concurrency)` bounds the number of concurrently running goroutines, a single stuck `FilterLogs()` call permanently consumes one of those slots, and once all slots are occupied by stalled requests the entire fetch — and therefore the exit certificate generation — halts with no progress and no error.

The impact is rated low because the consequence is a stalled process rather than incorrect output or loss of value, and the tool is an operator-run, one-shot utility. The likelihood is rated medium because reliance on a remote RPC endpoint that may become slow, rate-limited, or unresponsive over the large historical block ranges this tool scans is a realistic and recurring operational condition.

Recommendations

Apply a per-request timeout inside `fetchWindow()` so that an individual RPC call cannot block indefinitely, allowing the `errgroup` to observe an error, free the semaphore slot, and either retry or abort the sync. For example, derive a bounded context per call:


```
func (s *BridgeSyncerLite) fetchWindow(ctx context.Context, from, to uint64) ([]BridgeLeaf, error) {
    ctx, cancel := context.WithTimeout(ctx, s.cfg.RequestTimeout)
    defer cancel()

    logs, err := s.client.FilterLogs(ctx, ethereum.FilterQuery{
        FromBlock: new(big.Int).SetUint64(from),
        ToBlock:   new(big.Int).SetUint64(to),
        Addresses: []common.Address{s.cfg.BridgeAddr},
    })
    if err != nil {
        return nil, err
    }
    return classifyLogs(s.contract, logs, s.cfg.IgnoreUnsupportedL2Events, s.log)
}
```

Expose the timeout as a configurable field (with a sensible default) so operators can tune it for their RPC provider. Optionally, pair this with a bounded retry on timeout to improve resilience against transient slowness.

Resolution

The issue was acknowledged by the development team.

AET-22	AggregationVKeyHash Field Excluded from AggregationProofPublicValues.Hash()		
Assets	aggsender/types/fep_inputs.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `AggregationProofPublicValues.Hash()` method in `aggsender/types/fep_inputs.go:47` omits the `AggregationVKeyHash` field from its hash calculation, creating an inconsistent implementation pattern within the same file:

```
// aggsender/types/fep_inputs.go:18 - Struct has 8 fields
type AggregationProofPublicValues struct {
    L1Head          common.Hash
    L2PreRoot       common.Hash
    ClaimRoot       common.Hash
    L2BlockNumber   uint64
    RollupConfigHash common.Hash
    MultiBlockVKey   common.Hash
    TrustedSigner   common.Address
    AggregationVKeyHash common.Hash // 8th field - defined but not hashed
}

// aggsender/types/fep_inputs.go:63-70 - Hash method only includes 7 fields
args := abi.Arguments{
    {Type: tBytes32},
    {Type: tBytes32},
    {Type: tBytes32},
    {Type: tUint64},
    {Type: tBytes32},
    {Type: tBytes32},
    {Type: tAddress},
    // Missing: AggregationVKeyHash
}

packed, err := args.Pack(
    s.L1Head,
    s.L2PreRoot,
    s.ClaimRoot,
    s.L2BlockNumber,
    s.RollupConfigHash,
    s.MultiBlockVKey,
    s.TrustedSigner,
    // Missing: s.AggregationVKeyHash
)
```

The same file contains `AggchainParams.Hash()` which correctly includes all fields:

```
// aggsender/types/fep_inputs.go:144 - Includes AggregationVKeyHash
buf.Write(a.AggregationVKeyHash[:])
```

The issue was caused by an incomplete implementation during development - the 8th struct field was added but the corresponding hash method was not updated to include it in the ABI arguments and pack call.

Recommendations

Add the missing field to the hash calculation:

```
// Fix: Add 8th ABI argument type
args := abi.Arguments{
    {Type: tBytes32}, {Type: tBytes32}, {Type: tBytes32}, {Type: tUint64},
    {Type: tBytes32}, {Type: tBytes32}, {Type: tAddress},
    {Type: tBytes32}, // Add AggregationVKeyHash
}

// Fix: Add 8th field to pack call
packed, err := args.Pack(
    s.L1Head, s.L2PreRoot, s.ClaimRoot, s.L2BlockNumber,
    s.RollupConfigHash, s.MultiBlockVKey, s.TrustedSigner,
    s.AggregationVKeyHash, // Add missing field
)
```

Note: Verify this change is compatible with any on-chain contracts or external systems that validate the same hash, as this is a breaking change to the hash function.

Resolution

The issue was acknowledged by the development team.

AET-23	Anvil Prerequisite Check Does Not Respect <code>verifyNewLocalExitRootUsingShadowFork=false</code>		
Assets	tools/exit_certificate/step_check.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

When an operator explicitly disables shadow-fork verification by setting `verifyNewLocalExitRootUsingShadowFork=false`, the Step CHECK prerequisite guard still requires Anvil to be present in `$PATH` and aborts the entire pipeline if it is absent. This renders the offchain LER computation mode non-functional on machines without Foundry installed, even though no pipeline step uses Anvil in that configuration.

The Anvil check in `step_check.RunStepCheck()` calls `exec.LookPath("anvil")` unconditionally:

```
// step_check.go - RunStepCheck
if _, err := exec.LookPath("anvil"); err != nil {
    result.AnvilInstalled = false
    failures = append(failures, "anvil not found in $PATH (install from https://getfoundry.sh)") // @audit unconditional - ignores
    ↪ cfg.Options.VerifyNewLocalExitRootUsingShadowFork
}
```

In contrast, `step_g2.runStepG2()` correctly guards Anvil usage with `if cfg.Options.VerifyNewLocalExitRootUsingShadowFork`, so the prerequisite guard is stricter than the actual runtime dependency. The specification also documents the check as conditional: "required by Step G2 only when `options.verifyNewLocalExitRootUsingShadowFork=true`."

This issue has a low impact because the failure is loud and explicit (no silent data corruption or fund loss) and the workaround — installing the Foundry toolchain — is straightforward. This issue has a low likelihood because it only manifests when an operator explicitly sets the non-default `verifyNewLocalExitRootUsingShadowFork=false` AND lacks Anvil installed; the overall affected population is small given this is a niche operational tool, the option defaults to `true`, and operators may have Foundry installed from other work regardless.

Recommendations

Wrap the `exec.LookPath("anvil")` call in `step_check.RunStepCheck()` with `if cfg.Options.VerifyNewLocalExitRootUsingShadowFork`, mirroring the guard already used in `step_g2.runStepG2()`. When shadow-fork mode is disabled, skip the Anvil check entirely and log that it was not required.

Resolution

The issue was acknowledged by the development team.

AET-24	batchRPC() Response ID Mismatch Silently Produces Nil Results		
Assets	tools/exit_certificate/rpc.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`rpc.batchRPC()` silently returns nil result slots when the RPC server supplies out-of-range or duplicate response IDs, potentially causing downstream balance zeroing or address misclassification.

The function validates that the response count matches the request count, but does not validate individual response IDs. When a response ID falls outside the expected range, the code silently skips it:

```
for _, r := range responses {
    localIdx := r.ID - 1
    if localIdx < 0 || localIdx >= len(pendingIdxs) {
        continue // @audit silently skips - result slot stays nil, index not added to nextPending
    }
    origIdx := pendingIdxs[localIdx]
    // ...
}
pendingIdxs = nextPending
```

Because the skipped index is never added to `nextPending`, the final `len(pendingIdxs) > 0` check passes and the function returns without error, leaving nil slots in the results slice. A duplicate ID produces the same effect: one slot is overwritten and its partner remains nil.

Nil results propagate to callers: `step_b.unmarshalHexBigInt()` treats nil as zero (zeroing balances in `step_0.computeNativeBalance()`), and `step_b.isEOAResult()` treats nil as true (misclassifying contracts as EOAs in `step_b.classifyAddresses()`). The resulting certificate may omit SC-locked value or underreport EOA balances.

This issue has a low impact as an incorrect certificate could omit legitimate bridge exits, though in the default configuration Step F catches such mismatches by comparing certificate sums against the agglayer admin endpoint and aborts the pipeline before submission. This issue has a low likelihood as exploitation requires a compromised or non-conforming RPC endpoint.

Recommendations

Return an error from `rpc.batchRPC()` when any response ID falls outside the valid range or when duplicate IDs are detected. After processing all responses, verify that every result slot has been populated and treat any unresolved slot as a fatal error.

Resolution

The issue has been fixed in [PR#1716](#). The error is now propagated.

AET-25	classifyLogs() Relies On Implicit Correlated Initialisation Of contract		
Assets	tools/exit_certificate/bridgesyncerlite/downloader.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`parseBridgeEvent()` in the lite bridge syncer dereferences the `contract` parameter without a nil check, depending on an implicit invariant that the caller chain currently maintains but no part of the code enforces.

In `bridgesyncerlite/downloader.go`, `parseBridgeEvent()` calls into the contract binding immediately on entry:

```
func parseBridgeEvent(contract *agglayerbridge.Agglayerbridge, l types.Log) (BridgeLeaf, error) {
    event, err := contract.ParseBridgeEvent(l) // @audit nil contract panics with no upstream guard at this site
    // ... snip ...
}
```

The only call site, `classifyLogs()`, forwards `s.contract` from the downloader's state without validating it. The actual nil protection lives one frame further up in `fetchBridges()`, which guards on `s.client == nil`. `dialBridge()` atomically sets both `s.client` and `s.contract` together, so the client check implicitly covers the contract field — but this correlated-initialisation invariant is undocumented and entirely outside the visible call chain. `classifyLogs()` already nil-guards its `logger` parameter, demonstrating an inconsistent defensive posture, and the test suite exercises `classifyLogs(nil, &log)` with a nil contract, confirming that the interface does not enforce a non-nil receiver.

This issue has a low impact as it would crash the lite syncer process used by the exit certificate tool, not affect persisted state. This issue has a low likelihood because no current path decouples `s.client` from `s.contract`, and the trigger requires a future refactor that breaks the implicit correlation.

Recommendations

Add an explicit nil check inside `classifyLogs()` before any path that can reach `parseBridgeEvent()` — return an error when `contract == nil`. Alternatively, place the guard directly in `parseBridgeEvent()` itself. Either form converts the implicit correlated invariant into a self-documenting contract that survives future refactors.

Resolution

The issue was acknowledged by the development team.

AET-26	hexToUint64 Silent Failure On Invalid Input And Overflow		
Assets	tools/exit_certificate/hex.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `hexToUint64()` function in `hex.go` silently returns zero or a truncated value on empty, invalid, or overflowing hex input, with no error signal to callers. Unlike sibling functions `safeUint32()` and `safeUint8()` in the same file which return errors on overflow, `hexToUint64()` returns only a bare `uint64`. The `switch` statement lacks a `default` case, so invalid characters shift the accumulator without contributing bits, and strings exceeding 16 hex digits overflow silently.

```
func hexToUint64(s string) uint64 {
    s = strings.TrimPrefix(s, "0x")
    s = strings.TrimPrefix(s, "0X")
    var n uint64
    for _, c := range s {
        n <<= 4
        switch {
        case c >= '0' && c <= '9':
            n |= uint64(c - '0')
        case c >= 'a' && c <= 'f':
            n |= uint64(c - 'a' + hexLetterOffset)
        case c >= 'A' && c <= 'F':
            n |= uint64(c - 'A' + hexLetterOffset)
        }
    }
    return n
}
```

The most critical callers are `fetchL2ChainID()` in `step_sign.go`, where a malformed `eth_chainId` response produces `chainID=0` causing an invalid EIP-712 signature, and `resolveLatestBlock()` in `run.go`, where a malformed `eth_blockNumber` collapses deposit-scan ranges to zero so unclaimed L1 deposits are silently omitted. Both paths have downstream safety nets — the agglayer rejects the invalid signature, and Step F detects the resulting balance mismatch — limiting impact to confusing errors and wasted operator time. Exploitation requires a misconfigured or compromised RPC endpoint returning malformed hex responses, which is unlikely under normal operation.

Recommendations

Replace `hexToUint64()` with an error-returning variant that uses `strconv.ParseUint(trimmed, 16, 64)`, consistent with the existing `safeUint32()` and `safeUint8()` signatures. Update all call sites to propagate the returned error, using a logged warning with a zero sentinel for metadata-only uses such as `BlockNumber` fields in `L1Deposit` records.

Resolution

The issue has been fixed in [PR#1716](#). The error is now returned and propagated.

AET-27 Missing Nil Check On <code>r.discarded</code> In ERC-20 Probe Collect Callback			
Assets	tools/exit_certificate/step_b2.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The collect callback in `RunStepB2()` dereferences `r.discarded` without a nil guard, relying on an implicit invariant maintained elsewhere in the worker, which a future refactor could silently break.

The closure passed to `runWorkerPool()` decides which sub-list of ERC-20 results to append to by branching on `r.detected`, but it does not check the alternative pointer for nil:

```
func(r erc20ProbeResult) {
    if r.detected != nil {
        detected = append(detected, *r.detected)
    } else {
        discarded = append(discarded, *r.discarded) // @audit r.discarded dereferenced without nil check
    }
},
```

`detected` being nil does not in itself guarantee `discarded` is non-nil — they are independent pointer fields of `erc20ProbeResult`. The current safety net is twofold: the sibling worker function always sets exactly one of the two fields on a successful return, and `runWorkerPool()` only invokes the collect callback when the worker returned a nil error. Together these ensure the zero-value `erc20ProbeResult{}` never reaches the callback today. The vulnerability is the absence of an explicit guard at the dereference site itself — any future worker change that introduces a success path returning `erc20ProbeResult{}` with a nil error will trigger a nil pointer panic.

This issue has a low impact as it crashes a CLI run executed on operator infrastructure rather than a network service. This issue has a low likelihood as no current path produces such a result, and the bug only materialises under a future refactor of the worker function.

Recommendations

Add an explicit nil check before dereferencing in the collect callback. Alternatively, restructure `erc20ProbeResult` to use a discriminated-union pattern (for example, a `kind` enum) so the type system makes the invariant explicit and the dereference is statically safe.

Resolution

The issue has been acknowledged by the development team.

AET-28	Missing Nil Guard On GetCertificateHeader() Return In waitUntilFinal()		
Assets	tools/exit_certificate/step_wait.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `waitUntilFinal()` function can panic with a nil-pointer dereference if `client.GetCertificateHeader()` returns `(nil, nil)`.

After the error check, `waitUntilFinal()` unconditionally dereferences the returned `header` pointer:

```
header, err := client.GetCertificateHeader(ctx, certHash)
if err != nil {
    log.Warnf("GetCertificateHeader(%s) error (will retry): %v", certHash.Hex(), err)
    continue
}

// @audit - nil dereference if header is nil with err == nil
if header.Status != lastStatus {
```

The `AgglayerClientInterface` does not contractually guarantee a non-nil pointer when `err` is nil. A `(nil, nil)` return from the underlying gRPC client implementation would crash the tool. This issue has a low impact as it only affects a re-runnable CLI polling step and does not corrupt any submitted certificate state. This issue has a low likelihood as it requires the agglayer gRPC service to return a successful response with an absent `certificate_header` field, which is a non-standard condition.

Recommendations

Add a nil guard after the `GetCertificateHeader()` call in `waitUntilFinal()`, treating a nil header as a transient condition and continuing the polling loop.

Resolution

The issue has been acknowledged by the development team.

AET-29	Missing Nil Guard On <code>RPCExecutionError.Error()</code>		
Assets	tools/exit_certificate/rpc.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`(*RPCExecutionError).Error()` dereferences its pointer receiver without a nil check, exposing the exit certificate tool to a typed-nil-in-interface panic if a nil instance is ever returned through an `error` interface value.

In `tools/exit_certificate/rpc.go`, the `Error()` method reads `e.Data` immediately on entry with no upfront receiver check:

```
func (e *RPCExecutionError) Error() string {
    if e.Data != "" { // @audit dereferences e without nil check
        return fmt.Sprintf("RPC error: %s (data: %s)", e.Message, e.Data)
    }
    return fmt.Sprintf("RPC error: %s", e.Message)
}
```

When a nil `*RPCExecutionError` is stored in an `error` interface value, the interface itself is non-nil (it carries the concrete type), so the typical `if err != nil` guard does not catch it. Any caller that wraps such an error and later invokes `.Error()` on it — including standard logging paths and `fmt.Errorf("...: %w", err)` chains — would panic inside the `Error()` body.

This issue has a low impact because it would crash a CLI run rather than affect persisted state, and recovery is simply a re-run. This issue has a low likelihood because no current production path returns a typed-nil `*RPCExecutionError`; the bug is a latent defensiveness gap that a future refactor could expose.

Recommendations

Add a nil receiver guard at the top of `RPCExecutionError.Error()` that returns a sentinel string when the receiver is nil, for example `if e == nil { return "RPCExecutionError: nil" }`. This makes the implementation safe regardless of how the value is stored or passed through `error` interfaces.

Resolution

The issue has been acknowledged by the development team.

AET-30 Nil Pointer Dereference In sendGroup Debug Log Causes Panic On Native Bridge Exits			
Assets	tools/exit_certificate/step_g2.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `sendGroup` worker closure in `replayBridgeExits()` unconditionally dereferences `job.bridge.TokenInfo` and `job.bridge.Amount` in a `log.Debugf` call, causing a nil pointer dereference panic when the certificate contains a native bridge exit (which has nil `TokenInfo` by design). Go evaluates all function arguments before the call, so the dereference occurs regardless of log level — even without `-verbose`.

The same file already applies nil guards in `saveFailedExit()`, and `isNativeBridgeExit()` explicitly handles nil `TokenInfo`, confirming this is a recognised valid state. Any certificate that includes a native ETH exit triggers this panic in Step G2, terminating the shadow-fork replay mid-execution. This issue has a low impact and low likelihood because it causes a recoverable pipeline abort and only affects certificates containing native ETH exits in shadow-fork mode.

Recommendations

Add nil guards for `job.bridge.TokenInfo` and `job.bridge.Amount` before the `log.Debugf` call, matching the pattern already used in `saveFailedExit()`.

Resolution

The issue has been acknowledged by the development team.

AET-31 Nil Pointer Dereference On <code>null</code> JSON Array Element In Bridge Service Responses			
Assets	tools/exit_certificate/step_e.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

A malformed bridge service response containing a `null` element in its JSON array causes a nil pointer dereference panic, terminating the CLI process.

Both `fetchAggkitPendingBridges()` and `fetchZkevmPendingBridges()` deserialise HTTP responses into structs with pointer slices (`[]*aggkitBridgeEntry` and `[]*zkevmDeposit`). Go's `encoding/json` maps a JSON `null` array element to a nil pointer without returning an error. The subsequent iteration loops dereference each element without a nil guard:

```
// fetchAggkitPendingBridges()
for _, b := range result.Bridges {
    if b.DestinationNetwork == cfg.L2NetworkID { // @audit panics if b is nil
        matching = append(matching, b)
    }
}

// fetchZkevmPendingBridges()
for _, d := range result.Deposits {
    svcCounts[d.DepositCnt] = struct{}{} // @audit panics if d is nil
}
```

This issue has a low impact because the affected code path is an optional cross-check (only active when `BridgeServiceURL` is configured), runs within a CLI tool where a panic simply requires the operator to re-run the command, and no funds or state are at risk. This issue has a low likelihood because the bridge service is a trusted internal endpoint and a legitimate implementation will not return `null` array elements; triggering this requires either a severely broken service or active response manipulation.

Recommendations

Add nil guards before dereferencing each pointer element in both iteration loops, or change the struct field types from pointer slices (`[]*T`) to value slices (`[]T`) so that `null` elements decode as zero-value structs instead of nil pointers.

Resolution

The issue has been acknowledged by the development team.

AET-32	Non-Deterministic Map Iteration In <code>computeSCLocked()</code> Produces Run-To-Run Variation		
Assets	tools/exit_certificate/step_c.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`step_c.computeSCLocked()` iterates over a `map[string]LBTEEntry` to build the `SCLockedValues` slice, and Go randomises map iteration order at the language level. In the non-default offchain lite-tree mode (`verifyNewLocalExitRootUsingShadowFork=false`), this causes `step_g_events.buildLiteTreeFromCertificate()` to assign different positional `DepositCount` values to the same exits across runs, producing a different `NewLocalExitRoot` each time. The agglayer rejects a certificate whose root does not match its independent computation, so the practical consequence is a failed submission rather than a security breach.

```
for tokenKey, lbt := range lbtByToken { // @audit Go map iteration is randomised every run
    scLockedValues = append(scLockedValues, SCLockedValue{ ... })
}
```

In the default shadow-fork mode, `step_g_order.reorderCertificateByDepositCount()` restores a canonical order after Anvil replay, making the final certificate deterministic. The issue only manifests in a security-relevant output when an operator explicitly opts into offchain mode.

Recommendations

Replace the map range in `step_c.computeSCLocked()` with a sorted-key iteration to ensure deterministic output regardless of mode.

Resolution

The issue has been fixed in [PR#1711](#). A sorting feature was added.

AET-33	Non-deterministic Map Iteration In <code>RunStepB3()</code> Produces Non-Reproducible Certificate Outputs		
Assets	tools/exit_certificate/step_b3.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

In `exit_certificate.RunStepB3()`, the `Holders` slice is assembled by iterating over the `map[common.Address]*big.Int` returned by `fetchTokenBalances()`. Because Go randomises map iteration order, successive pipeline runs produce `ERC20HolderBreakdown.Holders` in different orders for identical onchain state, resulting in non-reproducible intermediate outputs (`step-b3-erc20-holders.json`). This inconsistency propagates through Step C and Step D into the certificate's `BridgeExits` ordering.

```
// step_b3.go - RunStepB3
holders := make([]ERC20Holder, 0, len(holderBalances))
for holderAddr, bal := range holderBalances { // @audit nondeterministic map iteration
    holders = append(holders, ERC20Holder{Address: holderAddr, Balance: bal.String()})
}
```

This issue has a low impact because each pipeline run produces an internally consistent certificate — the `NewLocalExitRoot` is computed from the same in-memory exit ordering, and the agglayer rebuilds the LER from the submitted certificate's own `bridge_exits` order, so no rejection occurs. The default shadow-fork mode (`verifyNewLocalExitRootUsingShadowFork=true`) additionally mitigates via `reorderCertificateByDepositCount()`, which deterministically sorts all exits by onchain deposit count after Anvil replay.

The practical consequence is limited to non-reproducibility of intermediate files across separate runs when using the offchain LER mode, which may complicate step-by-step recovery workflows. This issue has a low likelihood because it requires the non-default offchain mode, a non-empty `extraErc20Contracts` list with at least two holders, and an operator specifically re-running individual steps expecting byte-identical outputs.

Recommendations

Sort the `holders` slice by address bytes after the map iteration loop in `exit_certificate.RunStepB3()`, consistent with the sorting pattern already applied in `RunStepA()`. This ensures deterministic intermediate outputs regardless of the LER computation mode selected.

Resolution

The issue has been fixed in [PR#1711](#). A sorting feature was added.

AET-34	No Signal-Aware Context Allows Abrupt Termination Mid-Pipeline		
Assets	aggit/tools/exit_certificate/cmd/main.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The CLI is started with `app.Run(os.Args)` without ever wiring up a signal-aware context, and the downstream `Run()` entry point builds its context with `context.Background()`, which can never be cancelled. As a result, a `SIGINT` (Ctrl+C) or `SIGTERM` abruptly kills the process partway through the pipeline, with no opportunity to flush in-flight work or close resources cleanly, potentially leaving output files or the SQLite database in a partially written state.

The application is invoked here with no cancellation plumbing:

```
if err := app.Run(os.Args); err != nil { // @audit no signal-aware context passed into the pipeline
    _, _ = fmt.Fprintf(os.Stderr, "Error: %v\n", err)
    os.Exit(1)
}
```

Because `exit_certificate.Run()` derives its context from `context.Background()`, there is no propagation of an interrupt to the long-running steps (for example, the chain-scanning and tree-building phases). When the operator presses Ctrl+C, Go's default behaviour terminates the process immediately rather than unwinding gracefully, so any deferred cleanup, transaction rollback, or file finalisation that would normally run is skipped. The pipeline writes intermediate JSON artefacts (such as the per-step output files) and persists bridges to a SQLite DB, so an interruption at the wrong moment can leave a half-written file or DB on disk. A subsequent run that assumes those artefacts are complete could then operate on truncated or inconsistent input.

Both impact and likelihood are rated low. The likelihood is low because it requires the operator to interrupt the tool at the precise moment a file or DB write is in progress, and the tool is an operator-run, single-shot utility rather than a long-lived service. The impact is low because the failure is recoverable: the steps are designed to be re-run, and a corrupted or partial artefact can be regenerated by re-executing the affected step, so there is no permanent loss of funds or irrecoverable state — only wasted work and the risk of acting on a partial file if the corruption is not noticed.

Recommendations

Create a context that is cancelled on `SIGINT/SIGTERM` and thread it through the pipeline, replacing the use of `context.Background()` in `Run()`. The standard library provides `signal.NotifyContext()` for this:

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
// pass ctx down into the pipeline run
```

With a cancellable context propagated to each step, an interrupt will unwind the active operation, allowing deferred rollbacks and `Close()` calls to run and reducing the chance of leaving partially written output files or a partially written SQLite DB. Additionally, ensure that file writes are performed atomically (for example, write to a temporary file and rename on success) so that an interrupt can never leave a half-written artefact in place.

Resolution

The issue has been acknowledged by the development team.

AET-35	Offchain Step G2 Metadata Queries Use latest Block Instead Of Snapshot		
Assets	tools/exit_certificate/step_g2.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

In offchain mode (`verifyNewLocalExitRootUsingShadowFork=false`), Step G2 resolves token metadata and wrapped-token addresses against the live L2 chain at "latest" rather than at the Step G1 fork block, which can produce an incorrect `NewLocalExitRoot`.

When the shadow-fork is disabled, `runStepG2()` constructs a `metadataBackend` pointing directly at `cfg.L2RPCURL`:

```
metadataBackend = &anvilForkBackend{url: cfg.L2RPCURL, bridgeAddr: cfg.L2BridgeAddress}
```

This backend is then passed to `generateMetadata()`, which calls `ethCallBytes()` and `step_g2.callGetTokenWrappedAddress()`. Both functions hardcode "latest" as the block parameter:

```
// ethCallBytes - used by callGetTokenMetadata() and callGasTokenMetadata()
raw, err := singleRPC(ctx, rpcURL, "eth_call", []any{
    map[string]any{"to": bridgeAddr.Hex(), "data": "0x" + hex.EncodeToString(callData)},
    "latest", // @audit queries current state, not the G1 fork block
}, defaultRetries)
```

```
// callGetTokenWrappedAddress()
raw, err := singleRPC(ctx, anvilURL, "eth_call", []any{
    map[string]any{"to": bridgeAddr.Hex(), "data": "0x" + hex.EncodeToString(callData)},
    "latest", // @audit queries current state, not the G1 fork block
}, defaultRetries)
```

In shadow-fork mode this is benign because "latest" resolves within the Anvil fork's local state (pinned at the fork block). In offchain mode the queries hit the real L2, so any token metadata change (upgrade, redeployment) or sovereign-token mapping update occurring between the target block and the moment G2 runs will cause the computed leaf hashes to diverge from the intended snapshot, producing an incorrect `NewLocalExitRoot`.

This issue has a low impact because an incorrect `NewLocalExitRoot` would cause the certificate to fail agglayer verification, preventing settlement rather than enabling theft. This issue has a low likelihood because it only manifests in the non-default offchain mode and requires token metadata or bridge mappings to change in the window between the target block and G2 execution.

Recommendations

Pass the Step G1 fork block number to `ethCallBytes()` and `callGetTokenWrappedAddress()` as the `eth_call` block parameter when operating in offchain mode, replacing the hardcoded "latest" string with the hex-encoded fork block.

Resolution

The issue has been acknowledged by the development team.

AET-36	parseDecimalBigInt() Silently Returns Zero On Parse Failure		
Assets	tools/exit_certificate/hex.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Malformed decimal balance strings silently produce zero-value amounts in the exit certificate pipeline, potentially causing the tool to omit balances from the final certificate without surfacing an error to the operator.

The `parseDecimalBigInt()` function in `hex.go` parses a decimal string into a `*big.Int`. When `(*big.Int).SetString()` fails on a non-empty input, the function returns a zero-valued `*big.Int` with no error and no log message:

```
func parseDecimalBigInt(s string) *big.Int {
    if s == "" {
        return new(big.Int)
    }
    n, ok := new(big.Int).SetString(s, decimalBase)
    if !ok {
        return new(big.Int) // @audit silent zero - callers cannot distinguish from a genuine zero balance
    }
    return n
}
```

All eight call sites across `step_c.go` and `step_d.go` consume the return value without any indication that parsing failed. In `computeSCLocked()`, a zero `lbtBalance` causes the locked value to go negative and get clamped to zero, erasing the token from the SC-locked output. In `processBreakdowns()`, a zero `contractHolds` causes the entire holder breakdown for that token to be skipped. In the exit builders (`eoatToExits()`, `buildHolderBridgeExits()`, `buildSCLockedExits()`), zero amounts are filtered out and the corresponding `BridgeExit` is omitted from the certificate.

In practice, all balance strings that reach `parseDecimalBigInt()` originate from `big.Int.String()` in prior pipeline steps (Steps 0 and B), which always produces valid decimal output. The failure path would only trigger through filesystem corruption of intermediate JSON files or a similar data integrity failure between steps. This issue has a low impact as the affected balances would be silently omitted from the exit certificate in an emergency exit scenario. This issue has a low likelihood as it requires corruption of intermediate on-disk state that is produced by the tool itself.

Recommendations

Change `parseDecimalBigInt()` to return `(*big.Int, error)` and propagate parse failures to callers so the pipeline aborts on malformed input. Update all call sites to check the error and return it. At minimum, add a warning log when `ok` is false and the input is non-empty, allowing operators to detect corrupted intermediate data.

Resolution

The issue has been fixed in [PR#1716](#). The error is now returned and propagated.

AET-37	RPC Error Swallowed In Backward Scan Loop Produces Stale <code>L1InfoTreeLeafCount</code>		
Assets	tools/exit_certificate/step_i.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

A transient RPC failure during the backward block scan in `fetchL1InfoTreeLeafCount()` can cause the function to silently return a stale `L1InfoTreeLeafCount`, producing a certificate that the agglayer will reject.

The function scans L1 backwards in chunks, calling `queryUpdateL1InfoTreeV2()` for each range. When this call returns an error, the loop logs a warning and advances to the next older range rather than propagating the error:

```
leafCount, found, err := queryUpdateL1InfoTreeV2(ctx, cfg.L1RPCURL, cfg.L1GlobalExitRootAddress, start, end)
if err != nil {
    log.Warnf("eth_getLogs [%d-%d] error: %v", start, end, err) // @audit error is swallowed
} else if found {
    log.Infof("Found UpdateL1InfoTreeV2 at block range [%d-%d]: leafCount=%d", start, end, leafCount)
    return leafCount, nil
}

if start == 0 {
    break
}

end = start - 1
```

If the most recent `UpdateL1InfoTreeV2` event resides in the failed chunk but an older event exists in a subsequently-scanned chunk, the function returns the older event's `leafCount` with a nil error. The caller has no indication the result is stale.

A secondary issue is that context cancellation is also swallowed. When the context is cancelled, `queryUpdateL1InfoTreeV2()` returns `ctx.Err()`, but the loop treats it identically to any other error — logging and continuing through all remaining ranges before returning a misleading "no event found" error.

This issue has a low impact because the stale `L1InfoTreeLeafCount` causes the agglayer to reject the certificate rather than accept an invalid one, limiting the consequence to wasted operational time. This issue has a low likelihood because it requires a transient RPC failure specifically over the block range containing the most recent event, combined with an older event existing in a later-scanned range.

Recommendations

Propagate errors from `queryUpdateL1InfoTreeV2()` immediately rather than swallowing them. Additionally, check `ctx.Err()` at the top of each loop iteration to return the cancellation error without further iteration.

Resolution

The issue has been fixed in [PR#1716](#). The error is now returned and propagated.

AET-38	RunStepA2() Receipt Fetch Failures Silently Swallowed		
Assets	tools/exit_certificate/step_a.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`RunStepA2()` silently discards receipt fetch failures, providing no structured record of which transaction hashes could not be recovered after trace failures in Step A1.

When `receiptAddresses()` fails inside the worker pool, the error is logged as a warning but the worker returns `nil`, `nil`, which the pool treats as success. Unlike Step A1 — which persists `FailedTraces` to `step-a1-failed-traces.json` — `StepA2Result` has no equivalent field, and no output file records the failures. An operator cannot distinguish a fully successful A2 run from one where every receipt fetch failed.

```

addrs, fetchErr := receiptAddresses(ctx, cfg.L2RPCURL, hash)
if fetchErr != nil {
    log.Warnf("STEP A2: receipt failed for %s (skipping): %v", hash.Hex(), fetchErr)
    return nil, nil // @audit error swallowed - no structured failure accounting
}

```

This issue has a low impact because Step A2 is itself a degraded fallback — receipts inherently capture fewer addresses than `debug_traceTransaction` — and active addresses typically appear across multiple transactions, limiting the practical effect of missing a single receipt. The tool is also re-runnable against a healthy endpoint.

This issue has a low likelihood because `eth_getTransactionReceipt` is a widely supported, highly reliable RPC method that does not require an archive node, and `singleRPC()` already retries three times with exponential backoff before a failure reaches the swallow point. The scenario additionally requires `IgnoreOnTraceError=true` (non-default) to be active.

Recommendations

Add a `FailedReceipts []common.Hash` field to `StepA2Result` and persist it to `step-a2-failed-receipts.json`, mirroring the pattern established by Step A1's `FailedTraces`. Include the failure count in the completion log line.

Resolution

The issue was no longer relevant as of [PR#1721](#) because `RunStepA2()` was removed.

AET-39	saveJSON Silently Swallows File Write Errors		
Assets	tools/exit_certificate/run.go		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `saveJSON()` function silently discards file write errors, allowing the pipeline to continue with missing or stale intermediate data when disk I/O fails.

`saveJSON()` in `run.go` is declared with no return value. When `json.MarshalIndent()` or `os.WriteFile()` fails, the function logs the error at ERROR level and returns silently. All call sites (approximately 29 across the pipeline) treat the call as always successful and continue execution. If a subsequent step calls `loadJSON()` on the missing file, it either fails with a misleading "file not found" error or, if a stale file from a prior run exists, silently reads outdated data.

```
func saveJSON(dir, filename string, data any) {
    path := filepath.Join(dir, filename)
    content, err := json.MarshalIndent(data, "", " ")
    if err != nil {
        log.Errorf("Failed to marshal %s: %v", filename, err)
        return // @audit error swallowed - caller cannot detect failure
    }
    if err := os.WriteFile(path, content, filePermissions); err != nil {
        log.Errorf("Failed to write %s: %v", path, err)
        return // @audit error swallowed - caller cannot detect failure
    }
    log.Infof("Written: %s", path)
}
```

This issue has a low impact because the tool is a supervised CLI application where the operator is present and errors are logged at ERROR level. The scenario requires a disk-full condition combined with stale files from a prior run existing at the same path. This issue has a low likelihood because it requires a specific environmental failure (disk full, permission error) to occur during pipeline execution on a system that also has stale output files from a previous run.

Recommendations

Change `saveJSON()` to return an `error` and propagate it at every call site. All callers in the pipeline step functions should treat a failed write as a fatal pipeline error, for example: `if err := saveJSON(dir, fileStepAAddresses, combined); err != nil { return nil, fmt.Errorf("save step-a addresses: %w", err) }`.

Resolution

The issue has been fixed in [PR#1716](#). The error is now returned and propagated.

AET-40	scanBlockHeaders Silently Skips Blocks On JSON Unmarshal Error		
Assets	tools/exit_certificate/step_a.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

When `scanBlockHeaders()` fails to unmarshal a block-header JSON response, it logs a warning and continues to the next block, silently discarding all transaction hashes for that block. Any address exclusively touched in the skipped block will be absent from Step A's address set, never scanned in Step B, and never included in the exit certificate – resulting in permanent fund loss for those holders in this irreversible chain exit operation.

```
if err := json.Unmarshal(result, &bblock); err != nil {  
    log.Warnf("Failed to unmarshal block header: %v", err)  
    continue // block's transactions silently dropped  
}
```

This is inconsistent with the rest of Step A's error handling: the outer `RunStepA1()` loop propagates errors from both `scanBlockHeaders()` and `traceTransactions()`, and `traceOneTransaction()` returns unmarshal errors to callers. Only `scanBlockHeaders()` swallows the error. This issue has a medium impact and low likelihood because standard Ethereum nodes consistently return well-formed JSON, making the trigger condition (a non-standard node or data corruption) unlikely but the consequence (silent fund loss) severe.

Recommendations

Replace the `log.Warnf` and `continue` with a hard error return so that any unmarshal failure aborts Step A with a clear diagnostic rather than silently producing an incomplete address set. If graceful degradation is needed for test environments, introduce an explicit `ignoreBlockHeaderErrors` option mirroring the existing `IgnoreOnTraceError` pattern.

Resolution

The issue was no longer relevant as of [PR#1721](#) because `scanBlockHeaders()` was removed.

AET-41	uint64 Overflow In extractMetadata() Bypasses Bounds Check And Causes Runtime Panic		
Assets	tools/exit_certificate/step_e.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

A `uint64` arithmetic overflow in `extractMetadata()` causes the bounds-check guard to always evaluate to false when `metadataOffset` equals `MaxUint64`, allowing execution to reach a slice expression with a lower bound larger than the upper bound, triggering a Go runtime panic. The caller `decodeBridgeEvent()` extracts the metadata offset from bytes 192-223 of the log data using `big.Int.SetBytes(...).Uint64()`, which returns `MaxUint64` for the corresponding 32-byte ABI word. Inside `extractMetadata()`, the guard `metadataOffset+abiWordSize > uint64(len(data))` wraps to `31 > len(data)` which is false for any data of 32+ bytes (the minimum enforced by the caller is 256):

```
if metadataOffset+abiWordSize > uint64(len(data)) { // MaxUint64+32 wraps to 31
    return nil, nil
}
metadataLen := new(big.Int).SetBytes(
    data[metadataOffset : metadataOffset+abiWordSize], // PANIC: data[MaxUint64:31]
).Uint64()
```

An authentic bridge contract cannot emit such a value, so exploitation requires a compromised or malicious L1 RPC endpoint returning fabricated log data. The impact is low because the consequence is a CLI tool panic — not a funds-at-risk or data-corruption scenario — and the tool can simply be re-run against a correct RPC. Furthermore, the tool fundamentally trusts the L1 RPC for correctness of all returned data; a compromised RPC could cause far worse outcomes (e.g., silently hiding deposits or reporting them as claimed) without triggering any panic. The likelihood is low because it requires a compromised L1 RPC.

Recommendations

Replace the additive guard with an overflow-safe check: first verify `metadataOffset` does not exceed `uint64(len(data))`, then use subtraction on the validated values to confirm sufficient remaining bytes. Apply the same pattern to the second addition at `metadataStart + metadataLen`.

Resolution

The issue has been acknowledged by the development team.

AET-42 Unclaimed L1-to-L2 Message Deposits Are Excluded From The Exit Certificate			
Assets	tools/exit_certificate/step_e.go aggsender/converters/imported_bridge_exit_converter.go agglayer/types/types.go		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Step E allows the pipeline to continue when it finds unclaimed L1-to-L2 message deposits, but those deposits are not represented in the final certificate.

`RunStepE()` scans L1 `BridgeEvent` logs targeting the exiting L2, checks `isClaimed()` on the L2 bridge, and splits unclaimed deposits by leaf type:

```
unclaimed := filterUnclaimedDeposits(l1Deposits, claimedSet)
unclaimedAssets, unclaimedMessages := splitByLeafType(unclaimed)
```

Unclaimed assets are treated as blocking unless `ignoreUnclaimed=true`:

```
if len(unclaimedAssets) == 0 {
    return &StepEResult{FinalCertificate: certificate}, nil
}
if cfg.Options.IgnoreUnclaimed {
    return &StepEResult{FinalCertificate: certificate}, nil
}
return &StepEResult{FinalCertificate: nil}, fmt.Errorf("unclaimed deposits not supported ...")
```

Unclaimed messages, however, are only logged:

```
if len(unclaimedMessages) > 0 {
    log.Infof("Unclaimed message deposits (leaf_type=1, excluded from certificate): %d", len(unclaimedMessages))
}
```

They are saved in the Step E result but never merged into `certificate.ImportedBridgeExits`, and their presence does not cause an error. If there are no unclaimed assets, a message-only pending set reaches:

```
return &StepEResult{
    UnclaimedBridges: unclaimedAssets,
    UnclaimedMessages: unclaimedMessages,
    FinalCertificate: certificate,
}, nil
```

The rest of the pipeline does not catch the omission. Step F only groups and compares asset `BridgeExits` by token balance. Step G computes the new local exit root from the certificate's local `BridgeExits`. Neither step validates that pending L1-to-L2 message claims discovered by Step E were included, rejected, or intentionally waived.

The broader certificate stack does support message-type imported bridge exits. `ConvertBridgeExitFromClaim()` sets `LeafTypeMessage` when a claim is a message:

```
leafType := bridgetypes.LeafTypeAsset
if claim.IsMessage {
    leafType = bridgetypes.LeafTypeMessage
}
```

`agglayer/types.BridgeExit.Validate()` also accepts leaf types up to `LeafTypeMessage`, and `ImportedBridgeExit` is the certificate type used to carry claims from other networks. This means message claims are representable by the certificate model; Step E is silently discarding this class for the exit-tool path.

A concrete failure path is:

1. A user sends an L1-to-L2 bridge message to a contract on the exiting L2 before the exit target.
2. The message remains unclaimed on L2 at the target snapshot.
3. Step E finds the L1 `BridgeEvent` and `isClaimed()` returns false.
4. `splitByLeafType()` places the deposit in `unclaimedMessages`.
5. Because there are no unclaimed assets, Step E returns success with the certificate unchanged.
6. Step F passes because no token balance is missing.
7. Step G/H/I produce a final certificate that has no imported bridge exit for the message.

The impact is low. The check is a documented operational requirement — the operator must ensure all L2 bridge exits are settled before taking the exit snapshot. The pipeline fails early at Step H with a clear error message, before any irreversible action occurs. The operator's remediation is straightforward: wait for the agglayer to settle the outstanding certificate, then re-run the tool. No funds are at risk and no incorrect certificate can be produced.

The likelihood is low. Competent operators following the documented shutdown procedure would verify agglayer settlement status before halting the sequencer. The vulnerable window (between last settlement and halt) is operationally visible and the check exists precisely to catch this condition. The scenario requires the operator to skip the settlement verification step in the shutdown checklist.

Recommendations

Fail closed on unclaimed messages until proof support is implemented, using the same policy as unclaimed assets. If messages should be preserved, convert each unclaimed message into an `ImportedBridgeExit` with `LeafTypeMessage`, the correct global index, and claim proof data.

Minimal fail-closed guard:

```
if len(unclaimedMessages) > 0 && !cfg.Options.IgnoreUnclaimed {
    return StepEResult{
        UnclaimedBridges: unclaimedAssets,
        UnclaimedMessages: unclaimedMessages,
        FinalCertificate: nil,
    }, fmt.Errorf("unclaimed message deposits not supported: %d", len(unclaimedMessages))
}
```

Also include message deposits in the bridge-service cross-check, or document and require an explicit operator waiver when message claims are intentionally excluded.

Resolution

The issue has been acknowledged by the development team.

AET-43 Duplicate Sync() And AddBlocks() Methods With AddBlocks() Unused	
Assets	aggkit/tools/exit_certificate/bridgesyncerlite/syncer.go
Status	Closed: See Resolution
Rating	Informational

Description

The `Sync()` and `AddBlocks()` methods on `BridgeSyncerLite` have byte-for-byte identical implementations, and `AddBlocks()` has no production callers. This is a code-quality and maintainability issue with no direct security impact, but the redundancy invites divergence and confusion over which method to use.

Both methods fetch the bridge events for a block range and persist them, performing exactly the same work via `fetchBridges()` followed by `StoreBridges()`:

```
func (s *BridgeSyncerLite) Sync(ctx context.Context, fromBlock, toBlock uint64) error {
    bridges, err := s.fetchBridges(ctx, fromBlock, toBlock)
    if err != nil {
        return err
    }
    return s.StoreBridges(ctx, bridges)
}

func (s *BridgeSyncerLite) AddBlocks(ctx context.Context, fromBlock, toBlock uint64) error {
    // @audit identical body to Sync(); only the doc comment differs
    bridges, err := s.fetchBridges(ctx, fromBlock, toBlock)
    if err != nil {
        return err
    }
    return s.StoreBridges(ctx, bridges)
}
```

The only difference between the two is the documentation comment: `Sync()` is described as the initial full-history pass, while `AddBlocks()` is described as a follow-up pass for appending later ranges (for example, the shadow-fork blocks) before a single `BuildTree()` call. This is a purely semantic distinction that the code does not enforce. A search of the codebase shows that `AddBlocks()` is referenced only in `syncer_rpc_test.go`; the production pipeline uses `Sync()` (in `step_g1.go`), so `AddBlocks()` is effectively dead code outside of tests.

The impact and likelihood are both rated informational because there is no functional defect or security consequence: the two methods behave identically, so calling either one produces correct behaviour. The concern is maintainability — two identical methods can drift apart over time if one is later modified but not the other, and the redundant public surface area is mildly misleading to callers.

Recommendations

Consolidate the two methods into one. Either remove `AddBlocks()` entirely (since it is unused in production) and retain `Sync()`, or, if the semantic distinction is considered useful for readers, make `AddBlocks()` an explicit thin wrapper that calls `Sync()` so there is a single implementation:

```
func (s *BridgeSyncerLite) AddBlocks(ctx context.Context, fromBlock, toBlock uint64) error {
    return s.Sync(ctx, fromBlock, toBlock)
}
```

If `AddBlocks()` is removed, update `syncer_rpc_test.go` to call `Sync()` instead, preserving the existing test coverage for the second-range append scenario.

Resolution

The issue has been acknowledged by the development team.

AET-44 Incomplete --step Usage String Omits Valid Pipeline Steps

Assets aggkit/tools/exit_certificate/cmd/main.go

Status **Closed:** See [Resolution](#)

Rating **Informational**

Description

The usage text for the `--step` CLI flag lists only a subset of the available pipeline steps, omitting `h`, `i`, `submit`, and `wait`. A user relying solely on the `--help` output would have no indication that these documented, fully supported steps can be selected.

The flag is declared with a hard-coded usage string that has not been kept in sync with the actual set of steps the tool accepts:

```
cli.StringFlag{
  Name: "step",
  // @audit usage string omits steps h, i, submit and wait
  Usage: "Run a specific step: o, a, b, c, d, e, f, g, sign, or all",
  Value: "all",
},
```

This contradicts the application's own long-form `Description` in the same file, which documents steps `H`, `I`, `SUBMIT`, and `WAIT` in detail, and it is inconsistent with the `orderedSteps()` definition in `run.go` that actually implements them. The result is that a user who consults the concise `--step` help text — rather than reading the full description — would not learn that `--step h`, `--step i`, `--step submit`, or `--step wait` are valid. There is no functional or security impact: the omitted steps still execute correctly when invoked; the defect is purely one of documentation completeness and discoverability.

Both impact and likelihood are rated informational because the issue cannot cause incorrect behaviour, loss of funds, or an incorrect exit certificate — it only reduces the usability and accuracy of the CLI help. The worst realistic outcome is a user being unaware that a valid step exists and invoking the steps in a less convenient way.

Recommendations

Update the `--step` usage string to enumerate every step accepted by `orderedSteps()`, including `h`, `i`, `submit`, and `wait`, so the concise help matches the long-form `Description` and the implemented steps. To prevent this drift from recurring, consider generating the usage string from the canonical list of steps (for example, from the same source `orderedSteps()` is derived from) rather than maintaining it as a separate hard-coded literal.

Resolution

The issue has been acknowledged by the development team.

AET-45	isEOAResult() Lacks Explicit JSON Null Guard	
Assets	tools/exit_certificate/step_b.go	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `isEOAResult()` function does not explicitly handle a JSON `null` response from `eth_getCode`. When `json.Unmarshal([]byte("null"), &code)` is called on a non-pointer `string`, Go leaves `code` at its zero value `""` with no error, causing the function to return `true` and classify the address as an EOA. The codebase already guards against this pattern in `receiptAddresses()` in `step_a.go` using `string(result) == "null"`.

This issue is rated as informational as no standard Ethereum client returns `null` for `eth_getCode` (they return `"0x"`), making this a defensive coding inconsistency rather than a realistic failure mode.

Recommendations

Consider adding a `string(result) == "null"` guard to `isEOAResult()` that returns `false` (conservatively treating ambiguous responses as contracts), consistent with the existing pattern in `step_a.go`.

Resolution

The issue has been acknowledged by the development team.

AET-46	Nil Pointer Dereference In <code>finalizeStepFResult</code> When <code>certificate</code> Is Nil	
Assets	<code>tools/exit_certificate/step_f.go</code>	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `finalizeStepFResult()` function dereferences the `certificate` pointer without a nil guard, which would panic if a nil certificate were passed while `IgnoreBalanceMismatch` is enabled with mismatches present.

At `step_f.go` line [190], `capped := *certificate` copies the certificate struct value. This line is reached only when `allMatch` is false and `cfg.Options.IgnoreBalanceMismatch` is true. No nil check precedes this dereference. However, all callers in the normal pipeline always provide a non-nil certificate (Step D always returns a struct), making this reachable only through the exported API with an intentionally nil argument.

```
capped := *certificate // @audit panics if certificate == nil
capped.BridgeExits = capCertificateExits(certificate.BridgeExits, checks)
result.CappedCertificate = capped
```

This issue is rated as informational because it has no realistic trigger path in normal usage — all internal callers guarantee a non-nil certificate. It represents a defensive coding gap on an exported function's contract rather than a reachable bug.

Recommendations

Consider adding a nil guard at the entry of `finalizeStepFResult()` or `RunStepF()`: `if certificate == nil { return nil, errors.New("certificate must not be nil") }`.

Resolution

The issue has been acknowledged by the development team.

AET-47 Unbounded Polling Loops With No Timeout Or Signal Handling In Step WAIT	
Assets	<code>tools/exit_certificate/step_wait.go</code> <code>tools/exit_certificate/run.go</code>
Status	Closed: See Resolution
Rating	Informational

Description

Step WAIT's polling loops in `step_wait.waitUntilFinal()` and `step_wait.waitForVerifyBatchesOnL1()` have no maximum timeout and retry all RPC errors unconditionally, because the context originates from a bare `context.Background()` in `exit_certificate.Run()` with no deadline or cancellation wrapper. Additionally, `cmd/main.go` registers no OS signal handler, so SIGINT/SIGTERM bypass deferred statements.

This is rated as informational as the behaviour is by design for a CLI tool that explicitly waits for onchain settlement (which legitimately takes minutes to hours), no funds or data are at risk after certificate submission, and operators can terminate the process with Ctrl+C at any time.

Recommendations

Consider adding `signal.NotifyContext` in `cmd/main.go` to propagate SIGINT/SIGTERM as context cancellation, and consider exposing a configurable maximum wait timeout (e.g. `-timeout` flag or `waitTimeoutSeconds` config option) so automated pipelines can bound the polling duration.

Resolution

The issue has been acknowledged by the development team.

AET-48 Miscellaneous General Comments	
Assets	/*
Status	Closed: See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Invalid %w Verb And Missing Argument In log.Infof Calls Produce Garbled Output

Related Asset(s): `tools/exit_certificate/step_g2.go`

Two `log.Infof` calls in `step_g2.runStepG2()` contain format string errors: one uses `%w` (valid only in `fmt.Errorf`), producing `%!w(...)` output; the other has a `%d` placeholder with no corresponding argument, producing `%!d(MISSING)`. The error message also contains a typo ("mismath" instead of "mismatch"). Neither defect affects pipeline correctness — only the readability of log output.

Consider replacing `%w` with `%v` in the error log, supplying the exit count as the missing `%d` argument in the success log, and correcting the typo. Running `go vet` in CI would catch both format string classes automatically.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The issue has been acknowledged by the development team.

AET-49 Step A State Dump Completion Is Inferred From Exhausted Retries			
Assets	tools/exit_certificate/step_a.go		
Status	Open		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Step A can still silently accept a truncated state dump when the endpoint returns no new accounts across all three bounded re-seeks. PR #1729 correctly mitigates the observed premature empty cursor by seeking past the highest erigon account key, but `reseekPastFrontier()` still treats exhausted retries as proof that the trie ended:

```
tools/exit_certificate/step_a.go::reseekPastFrontier()
start := incrementKey(frontier)
for attempt := 0; attempt < accountRangeEmptyRetries; attempt++ {
    res, err := debugAccountRange(ctx, rpcURL, blockTag, start, accountRangePageSize, dialect)
    if err != nil {
        return nil, false, err
    }
    for key := range res.Accounts {
        if kb, ok := accountKeyBytes(key); ok && bytes.Compare(kb, frontier) > 0 {
            return res, false, nil
        }
    }
    if accountRangeEmptyRetryDelay > 0 {
        time.Sleep(accountRangeEmptyRetryDelay)
    }
}
// @audit a persistently faulty endpoint is indistinguishable from the genuine end of the trie
return nil, true, nil
```

The three calls are sampled over roughly one second and, after the final sleep, the caller records successful completion and returns the partial address set after roughly one and a half seconds. Neither recovery from a premature empty cursor nor retry exhaustion is logged. The `continue` taken on the re-seek path also skips the `accountRangeProgressInterval` progress log, so a run against a misbehaving endpoint goes quiet for long stretches of a multi-hour dump. The re-seek is currently sound because `cdk-erigon` iterates accounts in address order, whereas `geth` iterates in hashed-key order while returning addresses as map keys, making the highest observed address meaningless as a cursor there and the `dialectErigon` restriction necessary rather than conservative; that load-bearing assumption is not asserted in code or tests.

This issue has a low impact because the shipped re-seek genuinely resolves the observed failure mode, while Step F's default balance comparison aborts on the resulting native undercount. Omitted delivery can proceed with `ignoreBalanceMismatch=true`; alternatively, `nativeSCLockedFromContracts=false` folds missing holder value into the SC-locked amount, which is paid to `exitAddress` or left behind according to `skipSCLockedValue`. There is no demonstrated external theft path.

This issue has a low likelihood because it requires the endpoint fault to persist across all three attempts, whereas the observed mainnet fault was intermittent enough for the re-seek to recover.

Recommendations

Implement an independent completeness check, such as reconciling the summed native balance against a trusted invariant for the target network, before accepting retry exhaustion as the end of the trie. Fail closed when that check

does not pass.

If no completeness oracle is available, widen the retry envelope and apply exponential backoff.

Log every premature-empty recovery and exhausted retry sequence, and assert in code or tests that the erigon cursor advances in the same ordered key space as the returned account keys.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'